

Lecture 3: Execution Machinery

Rahul Krishna

Columbia University, COMS E6998-019, Fall 2026

Draft of September 25, 2026, for the meeting of September 25, 2026

Abstract

These notes explain how the software that runs an agent connects one step to the next. Extended state machines provide a way to describe the information an agent retains, the steps it can take, and the path followed by a run. Graphs and events provide two ways to implement these rules. We examine how the runtime updates state, selects and schedules steps, checks tool requests, and controls which actions may occur. We then explain how budgets limit work and how controlled experiments compare implementations.

Contents

1 Agents as state machines	1
1.1 Why a state machine describes an agent	1
1.2 Extended state machines	2
1.3 Transition notation	2
1.4 Behaviours, traces and computation trees	2
1.5 The planning agent as an extended state machine	3
1.6 Reading	4
2 What connects one step to the next	5
2.1 Storing the state machine's variables	5
2.2 Model context, runtime state, durable state	5
2.3 Reducers: combining updates with stored state	5
2.4 Field lifetimes	6
2.5 Invariants	6
2.6 State schemas of the loop and the planning agent	7
2.7 Reading	8
3 Routing, feedback, and stopping	8
3.1 Routing functions and guards	8
3.2 The same graph, different paths	9
3.3 Feedback edges	9
3.4 Loop bounds and termination	9
3.5 What a trace establishes	10
3.6 The demo: recorded endings and repairs	10
3.7 Reading	11
4 Scheduling, tools, and execution authority	11
4.1 Scheduling: when an update becomes visible	11
4.2 The tool contract	11
4.3 The dispatch cycle	12
4.4 Evidence of a request and its execution	12
4.5 Execution authority	13
4.6 The demo: scheduling and request refusal	13
4.7 Reading	14
5 An event-triggered implementation	14
5.1 Events, handlers and inversion of control	14
5.2 An event-triggered version of the machine	14
5.3 Two frameworks	15
5.4 The demo: the planning agent as a LlamaIndex workflow	15
5.5 Reading	16
6 Budgets and controlled measurement	16
6.1 Admission and accounting	16
6.2 A budget invariant	17
6.3 Exhaustion and truncation	18
6.4 Controlled measurement	18
6.5 Reading	19

A Conceptual exercises	21
A.1 How an agent moves through a task	21
A.2 What an agent remembers between steps	22
A.3 Choosing the next action and knowing when to stop	22
A.4 Tool requests, execution, and access	23
A.5 Events and queued work	24
A.6 Budgets and spending	25
B Answers and explanations	27
B.1 How an agent moves through a task	27
B.2 What an agent remembers between steps	27
B.3 Choosing the next action and knowing when to stop	28
B.4 Tool requests, execution, and access	29
B.5 Events and queued work	30
B.6 Budgets and spending	31

1 Agents as state machines

1.1 Why a state machine describes an agent

A state machine describes how an agent proceeds through a task. It leaves out the details of prompts, model internals and tool code so that we can study the rules governing execution.

An agent works in separate steps, such as calling a model, running a tool or checking a result. It retains information between steps and may receive new inputs. The program specifies how this information can change and which step can follow. These are the elements of a state machine: i) a set of states, ii) inputs received during a step, iii) outputs produced by a step, iv) an initial configuration, and v) a transition rule that specifies the next state and output from the current state and input. A workflow that is meant to end also defines terminal configurations. Once it reaches one, the runtime schedules no further step.

Definition 1.1 (State). State: “condition of the system at a particular point in time”. It “[e]ncodes everything about the past that influences the system’s reaction to current input” [18, slide 10].

State is the information the program retains between steps. It must preserve the facts about earlier steps that can affect what happens next. For example, the controller reads its instructions and earlier observations, while the runtime may check how many model calls have already been made. Two visits to the same controller can therefore lead to different next steps. The code is unchanged, but the information it reads has changed.

What counts as an input depends on where we draw the system boundary. In our description, model replies and tool results arrive during a step; the retained information alone does not determine them. We can include workspace files in the environment we model, or treat a tool’s report about those files as an input. In either case, **the retained information constrains the permitted transitions. It does not determine which external result arrives.**

Definition 1.2 (Determinism). Determinism: “In every state, for all input values, exactly one (possibly implicit) transition is enabled” [18, slide 29].

The system boundary also determines whether we describe the agent as deterministic. If we treat each model reply as an input, routing is deterministic when the updated state selects exactly

one outgoing edge. The same state and reply then select the same edge. If we instead include the model inside the system and leave its internal choice unspecified, the same retained information can lead to either a tool call or a submission. This is a nondeterministic description: it represents choices whose causes we have left out of the model.

Neither description assigns probabilities to replies. A probabilistic machine assigns a probability to each transition. For a nondeterministic machine, “no such probability is known. We just know that any of the enabled transitions from a state can be taken” [18, slides 33 and 37]. Our descriptions specify what can happen without estimating how likely it is. Deterministic routing alone does not make a run reproducible. Reproducing a run also requires the same inputs, including the model replies.

1.2 Extended state machines

An agent usually has a few named steps, called *control locations*, but retains many possible values in its conversation, counters and other data. In a plain state machine, each combination of a location and those values would need a separate state. The resulting diagram would make the sequence of steps difficult to see. An *extended finite state machine* (EFSM) separates the finite set of control locations from the values stored in variables.

Definition 1.3 (Extended state machine). “Extended state machines augment the FSM model with variables that may be read or written” [19, slide 4].

Definition 1.4 (Configuration). A configuration of an extended state machine is a pair (q, v) . The control location q identifies the current step and appears as a node in the diagram. The valuation v gives the current value of every declared variable.

For our agent, the control locations include planning, calling the controller, running tools, checking a candidate and ending the run. Variables hold the conversation, observations, plan, counters and status. For example, `(controller, model_calls=2, ...)` and `(controller, model_calls=5, ...)` are different configurations at the same location. Both execute the controller code, using different data. The extended state machine shows this shared step once and records the differences in its variables.

A finite number of locations does not guarantee a finite number of configurations. Consider a traffic-light controller at a pedestrian crossing. It has four locations and one variable, *count*, with values in $\{0, \dots, 60\}$. The controller therefore has at most $4 \times 61 = 244$ configurations. A message list with no maximum length has no comparable limit. **The control locations are finite; the configurations need not be.** A procedure that checks safety by listing every reachable configuration may therefore never finish. The diagram can still have a finite number of locations and transition rules.

1.3 Transition notation

A transition can have three parts: a *guard*, an *output action* and a *set action*. The guard is a condition that must be true for the transition to occur. The output action specifies what the transition produces, and the set action specifies how it updates the variables. We write *guard / output action* above the edge and the set action below it. For example, the traffic-light controller’s red-to-green edge has *count* ≥ 60 / *sigG* above *count* $:= 0$. The initial arrow specifies the variables’ starting values. We declare variables, inputs and outputs separately, following the notation in Seshia [19, slides 5–6]: “We make explicit declarations of variables, inputs, and outputs to help distinguish the three.”

We also use a compact notation within the text. Starting from

configuration (q, v) , a transition reads input e , tests guard g , produces output *out* and applies the update *set*:

$$(q, v) \xrightarrow{e; g / \text{out}; \text{set}} (q', v').$$

The guard $g(v, e)$ is a predicate: a condition that is true or false for the given variables and input. The set action changes v to v' . Table 1 relates each element to an agent.

Element	Symbol	In an agent
Control location	q	The computation ready to run: a model call, a tool dispatch, a check. There are finitely many, each shown as one node.
Variables	v	What the program retains between steps: messages, observations, a plan, counters, check results, a status.
Input	e	A result that arrives during a step: a model reply, a tool result, a check result.
Guard	$g(v, e)$	A condition on the variables and input, such as <i>the reply requests a tool</i> or <i>the call budget is exhausted</i> .
Output action	<i>out</i>	A request to perform an operation, or a reported terminal outcome.
Set action	<i>set</i>	Store a reply, extend the observations, increment a counter, replace a plan.
Initial configuration	(q_0, v_0)	The first location, with the initial set action: opening messages present, counters at zero, no observations, status running.
Terminal configuration		A configuration from which no further computation is scheduled; the record carries the reason for ending.

Table 1: The elements of an extended state machine, interpreted for an agent. Adapted from the general notation for extended state machines.

Consider a controller step. The call count c is below the budget B , so the controller is allowed to call the model. The reply e contains a tool request. In the transition below, the reply is the input, the guard checks for tool calls, and the output action requests their execution. The set action adds the reply to the messages H and increases c by one. The next location is *tools*:

$$(\text{controller}, v) \xrightarrow{e; \text{calls}(e) \neq \emptyset / \text{request}(e); H := H \cdot e, c := c + 1} (\text{tools}, v').$$

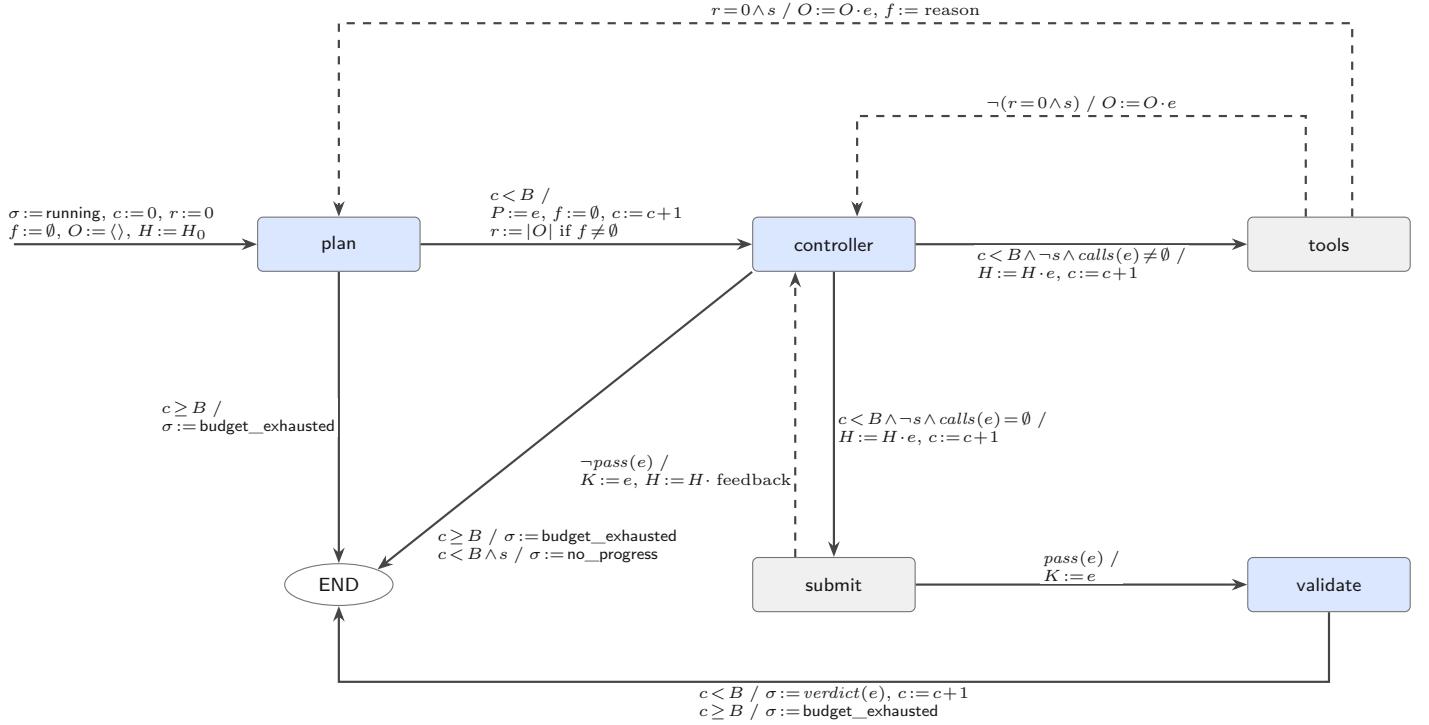
When the tools return their results, the next transition adds those results to the observations and can return to the controller. The controller now has more messages H , a larger call count c , and new observations. It is at the same location with a different configuration, so it can make progress while repeating the same step.

1.4 Behaviours, traces and computation trees

A transition describes one permitted step; a run follows a sequence of steps. The following definitions distinguish a sequence from its recorded history and from the alternatives the machine permits [19, slide 8].

Definition 1.5 (Behaviour). “FSM behavior is a sequence of (non-stuttering) steps”.

Definition 1.6 (Trace). “A trace is the record of inputs, states, and outputs in a behavior”.



Symbol	Meaning	Symbol	Meaning
e	The step's input: a reply, tool results, or check results.	O	Observations.
σ	Status.	P	Plan.
c	Model calls completed.	K	Check results.
B	Call budget (40).	f	Failed step (failed_step).
H	Messages.	r	Number of observations at the last revision (replanned_at).
H_0	The opening messages.	s	The last three observations since index r , including any just appended, are identical, and none is a successful edit.
$calls(e)$	The tool calls in reply e .	$pass(e)$	All three checks pass.
$verdict(e)$	accepted or rejected.		

Figure 1: The planning agent as an extended state machine, drawn from its `graph.py`. Each edge is labelled *guard / set action* in the notation of Section 1.3; output actions are omitted. Labels are stacked where two transitions share an arrow. Blue locations call a model; grey locations run tools or checks. The table under the drawing defines every symbol on the edges.

Definition 1.7 (Computation tree). “A computation tree is a graphical representation of all possible traces”.

A stuttering step has no input, changes no state and produces no output. A behaviour leaves out these steps. For a fixed input sequence, “A deterministic system exhibits a single behavior”, while “A non-deterministic system exhibits a set of behaviors” [18, slides 29 and 36]. A computation tree shows the possible sequences, branching when more than one transition can occur.

A trajectory is one path through the computation tree. It starts at the root and follows the run to a terminal configuration or to the point where execution stopped. Each visit to a control location appears as a separate tree node. Four visits to the controller therefore appear as four nodes, each with its own configuration and possible next steps. The machine diagram shows which transitions are permitted. The tree shows how those transitions can follow one another from the initial configuration.

An agent’s event log provides a practical trace. During execution, the agent holds its current state in memory. When the process ends, unsaved state is lost, while logs and workspace files can re-

main. **State holds the information available at the current step; the trace records what happened during the run.** Our agent logs six kinds of event: `tool_request`, `tool_result`, `model_call`, `routing`, `state_change` and `terminal`.

An event log usually contains less information than the trace in Definition 1.6. It records selected inputs and outputs, with only some state values. Copying every variable at every step would be expensive. To reconstruct a trajectory, we therefore combine the recorded facts with what the transition rules imply. When describing a run, we must distinguish these two sources of evidence.

1.5 The planning agent as an extended state machine

Our running example is the *planning agent*. This name refers to the complete workflow; *planner* refers to its plan node and the model role used there. The agent receives a scanner alert and drafts a plan. The controller then alternates between model calls and tool calls to carry out the work. It submits a candidate patch to three automatic checks, after which a validator decides whether to accept it.

In Figure 1, each transition includes completing a step, updating the variables and selecting the next location. The diagram leaves out the internal operations of model and tool calls. It describes

the version used for the two recorded runs. The current code also has `scan` and `normalize` nodes before `plan`.

The initial arrow enters `plan` with $\sigma := \text{running}$, no model calls, no observations, no revision ($r = 0$, $f = \emptyset$), and the opening system and task messages H_0 . Figure 1 shows the following routes.

- **Planning.** `plan` makes one model call, stores the proposed plan in P , and moves to `controller`.
- **Acting.** A controller reply with tool calls, $\text{calls}(e) \neq \emptyset$, routes to `tools`, which appends the results to O . Unless the stall guard holds, control returns to `controller`.
- **Revising.** The stall guard $r = 0 \wedge s$ checks two conditions: the plan has not yet been revised, and the last three observations are identical with no successful edit among them. If both hold, `tools` stores the reason in f (`failed_step`) and returns to `plan`. That node replaces P , clears f , and sets $r := |O|$ (`replanned_at`). This records where the new plan’s observations begin. Later checks of s use only observations made after the revision.
- **Submitting.** A reply without tool calls, $\text{calls}(e) = \emptyset$, selects `submit`. This node checks that the patch applies, changes the flagged line, and leaves no alert on a rescan. If a check fails ($\neg \text{pass}(e)$), it adds feedback to H and returns to `controller`. If all checks pass ($\text{pass}(e)$), it proceeds to `validate`.
- **Judging.** After its model call, `validate` moves to `END`. It sets $\sigma := \text{verdict}(e)$, either `accepted` or `rejected`. If the reply contains no decision, the runtime records a rejection and identifies itself as the source.
- **Exhaustion.** Before a model call, `plan`, `controller` and `validate` each test $c < B$, with $B = 40$. If the test fails, the run moves to `END` with $\sigma := \text{budget_exhausted}$. At the controller, s can hold only after a plan revision. When it holds, the run ends with $\sigma := \text{no_progress}$. The routing function after `submit` also checks the status, although no step on the path to `submit` changes it.

The planner stores subgoals as a list in a variable. Adding a subgoal leaves the control locations unchanged, and routing does not check which subgoal is current. **The plan is data read by the controller; its subgoals are not control locations.** The controller receives the plan in a message, so the plan affects execution through the controller’s replies.

Our worked run, `bc284c6b`, repairs a Bandit B608 alert (SQL injection) at line 46 of `testcode/BenchmarkTest00283.py`. Its log records this trajectory, with the tool each `tools` step ran:

```
plan
-> controller -> tools(read_file)
-> controller -> tools(search)
-> controller -> tools(read_file)
-> controller -> tools(edit)
-> controller -> submit -> validate -> END(accepted)
```

The trajectory contains 25 events. Events 1–2 record the model call in `plan` and the plan it stored. Events 3–18 record four tool rounds. Each contains a controller call, routing decision, tool request and tool result. The requests are `read_file` on the flagged file, `search` in `helpers/db_sqlite.py`, `read_file` on that helper, and `edit` on the flagged file. Events 19–22 are submission: a controller reply with no tool calls, the route to `submit`, three passed checks, and the route to `validate`. Events 23–25 record validation and the terminal event, `accepted`. The run made seven model calls: one by `plan`, five by `controller` and one by `validate`. It logged 12,693 input and 3,755 output tokens.

Figure 2 shows the beginning and end of this trajectory as a path through a tree. Each visit to `controller` appears as a new node. Its reply selects either `tools` or `submit`. At `submit`, the checks determine the branch; at `validate`, the verdict does. The

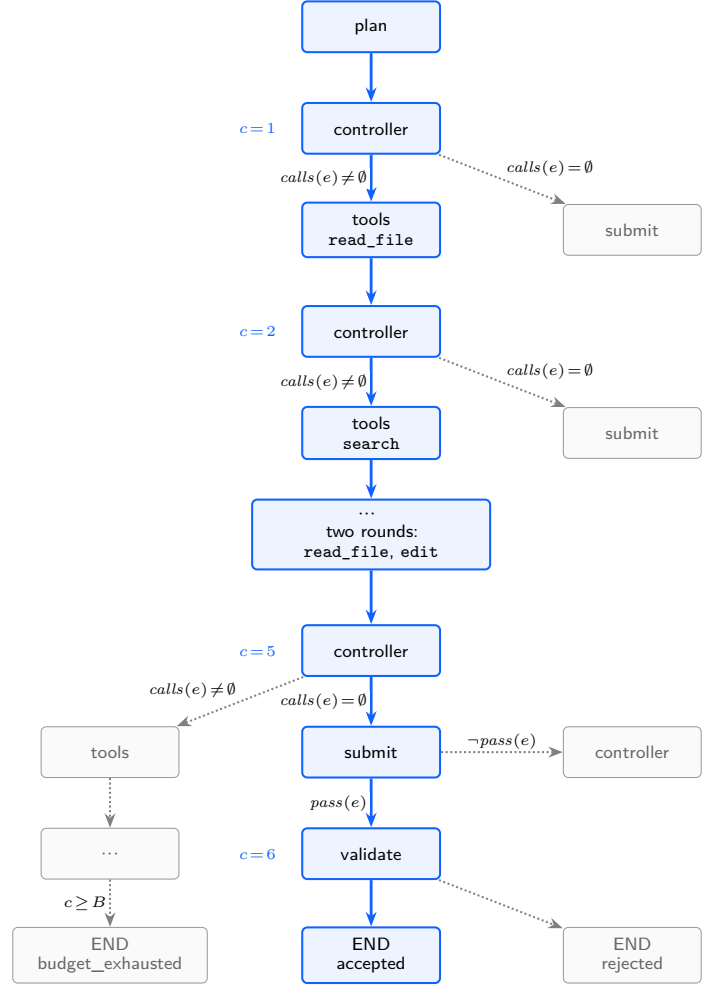


Figure 2: A fragment of the planning agent’s computation tree around run `bc284c6b`. Blue nodes and edges show the recorded trajectory; the `...` node omits two controller and tool rounds. Dotted grey branches are continuations the guards permit under other replies or check results; the grey `...` omits further rounds. c is the number of model calls completed on reaching a node, derived from the log and the set actions. No probabilities are implied.

tree includes a budget-exhaustion branch only where the guard allows it. In the recorded run, c never exceeds 7, so ending with `budget_exhausted` would require more rounds. For readability, the figure omits branches where tool results trigger the stall guard.

The log records only some facts about this run. A successful `tool_result` reports success and the result’s length, but leaves out the observation itself. The returns from `tools` and `plan` to `controller` have no routing events. We infer them from the next controller call and the permitted transitions. The log also omits the counters. Reconstructing the trajectory therefore requires the log and the transition rules together. Recovering every variable’s value at every step would require a more detailed log.

1.6 Reading

Seshia [18], slides 10–37, on state, transitions and guards, determinism and receptiveness, and nondeterministic machines and their computation trees; and Seshia [19], slides 4–8, on extended state machines, their notation, the traffic-light example, and behaviours and traces.

Looking ahead. The machine has fixed control locations and variables whose values change at each step. The next section describes the record that holds those variables, the steps that may update each field, and the rules for merging updates with stored values.

2 What connects one step to the next

2.1 Storing the state machine’s variables

The machine diagram shows which step may follow another. A shared state record holds the information those steps pass to one another. Routing and dispatch, discussed later, also use this record.

A configuration has two parts: the control location and the values of the variables (Definition 1.4). The runtime represents these parts separately. The state record holds the variable values, while the runtime tracks which step runs next. A *state schema* specifies the name and type of each field in the record. Each step reads the record and returns updates for the fields it changes. In our code, `StateGraph(AgentState)` creates a graph that uses `AgentState` as its schema.

Updating state involves two operations. First, a node reads the current state s_t , obtains an external result e_t , and returns a partial update u_t . The runtime then combines that update with the stored state, one field at a time:

$$u_t = \text{node}(s_t, e_t), \quad s_{t+1} = \text{merge}(s_t, u_t).$$

Here e_t represents a model reply or tool observation obtained while the node runs. In the code, the node receives state as its argument and obtains this external result itself. Any field missing from the update keeps its previous value in s_{t+1} .

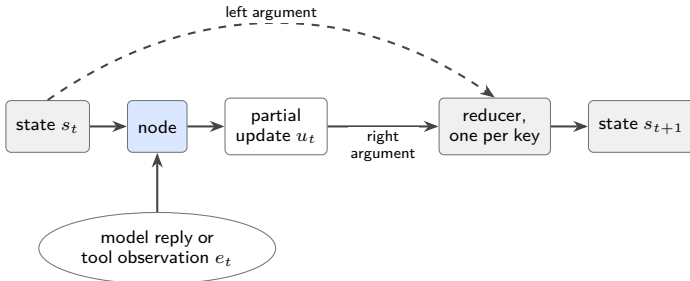


Figure 3: A state update. The node returns changes, and the runtime combines them with the stored state. For each updated field, the reducer takes the stored value as its left argument and the node’s contribution as its right argument.

A node can return just the fields it updates. As the LangGraph documentation states, “the Node does not need to return the whole State schema - just an update” [9]. After a model call, our controller returns three of the loop’s ten fields: `messages`, `model_calls` and `usage`. Either of its two early-stop paths returns only `status` (Figure 4).

The runtime applies each field’s merge rule and stores the result. The node therefore returns its contribution, while the merge rule determines whether to append it, add it to a total or replace the previous value.

2.2 Model context, runtime state, durable state

The information available to a model call overlaps with the information retained by the runtime. The model receives the instructions and data assembled for that call. The runtime also keeps counters and other fields used to control execution. State is *durable* when it is saved beyond the life of the process; resuming a run requires enough saved information to continue from the stopping point.

Counters provide an example of information the runtime uses without sending it to the model. Our loop refuses a model call when `model_calls` reaches forty, or when the last three observations are identical. The model receives neither limit. The runtime enforces them by checking the state before the call. Including a limit in a prompt would not replace this check.

Record	What it holds, and who reads it
Model context	Instructions, selected history and observations sent in one model call. Different model roles can receive different contexts. Our controller receives the conversation; the plan node and validator receive inputs prepared for their roles.
Runtime state	The record available between steps. It includes fields that are never sent to the model. Nodes, routing functions and limit checks read it.
Durable state	State saved so that it remains available after the process ends. To resume a run, the saved record must also correspond to the external state, such as workspace files, and identify pending work.

Table 2: Three kinds of record. Each call’s model context is derived from runtime state. Durable state is runtime state saved beyond the process lifetime.

If our agent’s process stops during a run, a new process cannot recover the state held in its memory. Neither graph uses a checkpoint to save that state. Trace files and workspace files may remain, but they are insufficient to resume the run. LangGraph allows a graph to use a checkpoint, which gives “agents short-term memory” and “lets LangGraph applications keep useful information beyond a single graph run” [10]. Our JSONL trace contains selected events (Section 1.4). **The saved trace does not contain enough information to resume the run.**

Saving state does not by itself prevent repeated work after a restart. The runtime may need to rerun an interrupted node from the last saved point. That node could repeat a file change or a paid call if its completion was not saved. The amount of repeated work depends on where the runtime places checkpoints and task boundaries. Later units examine these recovery requirements. Here we focus on the state needed while a run is executing.

2.3 Reducers: combining updates with stored state

A field can retain a history, a running total or just its latest value. Its *merge function* determines how each update changes the stored value, and therefore what later steps can read.

In an extended state machine, a transition’s set action assigns values to variables (19, slide 5). Our implementation divides this action between the node and the runtime: the node returns an update, and the runtime combines it with the stored value.

Definition 2.1 (Partial update). A partial update is a step’s returned contribution to state. It gives values for some fields and leaves the others unchanged.

Definition 2.2 (Merge function). A merge function combines a field’s stored value with the value supplied in an update. Its result becomes the field’s new value. If no merge function is specified, the update replaces the stored value.

LangGraph calls the merge function a *reducer*. We use that term from here on. A reducer takes two arguments, (`current`, `update`) $\rightarrow$ `new`, and is attached to a field through `Annotated[T, fn]`. When a node returns that field, the runtime applies its reducer. For example, `add_messages` appends new messages and replaces an existing message when an update uses the same identifier. Other functions of two arguments can also serve as reducers, including Python’s `operator.add` and functions defined in our code.

Most fields use one of three merge rules.

- **Append.** Add the new entries to the end of the stored list. Keeping earlier entries allows later steps to examine a

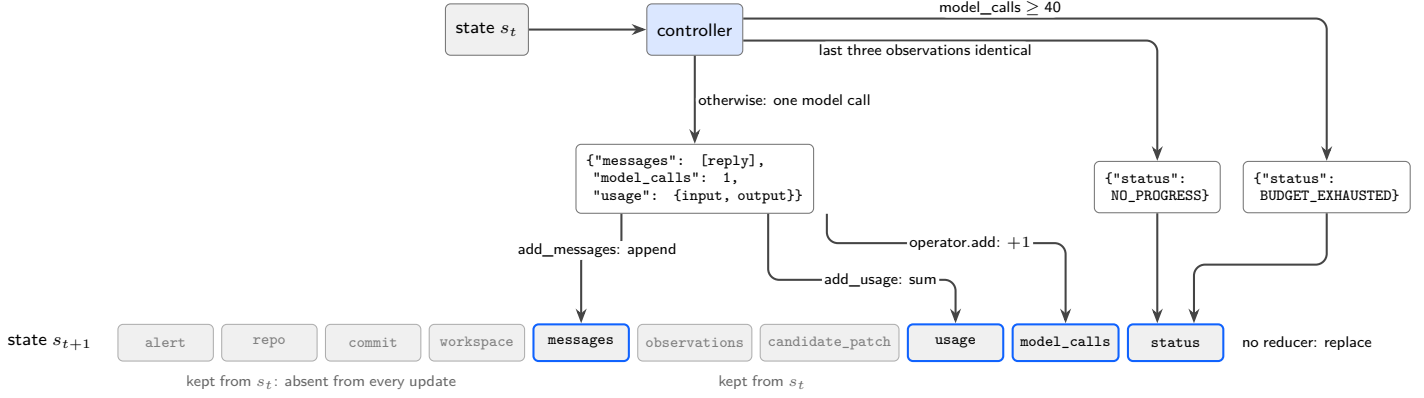


Figure 4: The controller’s three possible updates to the loop’s ten-field state record. Guards select the path. After a model call, the controller returns three fields; each early-stop path returns one. Arrows name the reducers used for those fields. No reducer is specified for `status`, so an update replaces its stored value. The six fields without arrows are absent from all three updates and retain their values from s_t .

sequence, such as checking whether the same result appeared three times in a row. Keeping only the latest result would lose that information.

- **Replace.** Store the new value in place of the previous one. This default rule suits fields such as the current plan, decision or status, where later steps need the latest value.
- **Accumulate.** Combine the new contribution with the stored value, usually by addition. Each node reports only its own contribution, such as the cost of its model call. The reducer adds that contribution to the running total.

The merge rule also determines how to clear a field. Appending an empty list adds nothing: combining `[]` with `[x, y]` still gives `[x, y]`. **An empty update does not clear a list that uses an append reducer.** To clear it, we must bypass the reducer and replace the stored value. LangGraph provides `Overwrite` for this purpose. Recording the reset as a separate trace event makes this exception to the normal merge rule visible.

2.4 Field lifetimes

A reducer specifies how to combine an update with a stored value. We must also decide how long that value remains valid. Later steps can make wrong decisions if they rely on an outdated value.

A field’s *lifetime* is the part of a run during which other steps can rely on its value. It begins when the value is written and ends when an event makes it outdated. The writer must then replace or clear it. We choose this lifetime when designing the field. The reducer alone cannot enforce it; the nodes that write the field must update or clear it at the appropriate time.

For each field, identify the step that writes it and the event that makes it outdated. The following six lifetimes describe common choices.

- **Identity.** Set the field before the run and leave it unchanged. No custom reducer is needed. Task details, the target and the workspace usually have this lifetime because they define what the run is doing and where it may act. Changing them would change the task during execution.
- **Run.** Keep the information throughout the run and add to it as work proceeds. A history uses an append reducer; a running total uses an accumulating reducer. Conversations, observations, and call or token totals have this lifetime when later steps need the full history or total.
- **Current.** Keep the latest value until the step that writes it supplies a new one. The default replace rule suits a current plan, candidate, check result or status.
- **Handoff.** One node writes information for another node to read once and then clear. For example, it can record why the

planner must revise a plan. Leaving the field set can cause a later routing decision to act on the same signal again.

- **Window.** Keep the full history, but store an index marking where a particular check should start reading. This lets a check use only recent events while other steps retain access to earlier ones. Updating the index moves the window without deleting history.
- **Attempt.** Keep the value for one attempt and clear it before starting another. This applies to a failed attempt’s candidate and check results when they would mislead the next attempt. The merge rule must allow clearing: a replacement field can receive an empty value, while a field with an append reducer needs `Overwrite`.

The choice of lifetime matters for counters that enforce limits. A limit covering several attempts needs a counter that persists for the whole run. **Resetting a counter after each failure prevents it from enforcing a limit across attempts.**

2.5 Invariants

A spending limit must hold throughout the run. To prove that it does, we must examine all transitions that could affect it. Specifying a reducer or lifetime for one field is only part of that argument.

Definition 2.3 (Invariant). “An invariant for a state machine is a predicate, P , on states, such that whenever $P(q)$ is true of a state, q , and $q \rightarrow r$ for some state, r , then $P(r)$ holds” [17, p. 5].

For an extended state machine, each state in this definition is a configuration (q, v) , and the arrow represents one transition. The proof uses induction: show that the condition holds at the start, then show that any transition from a configuration satisfying it leads to another configuration satisfying it. It follows that the condition holds throughout the run.

The second step considers every configuration that satisfies the condition, even those the program never reaches. Sometimes the property we want is too weak to complete this step. We then prove a stronger condition that includes it. The following example shows why.

We want to show that *model calls stay within the call budget*. Let B be the budget, c the call count stored by the program, and k the actual number of completed calls. The runtime can read only c . We use k in the proof to describe what actually happened. Our goal is $k \leq B$. We prove this by also showing that the stored count matches the actual count, using $P \equiv (c = k \wedge c \leq B)$. At the start, no call has been made and $c = 0$, so P holds. Each transition must

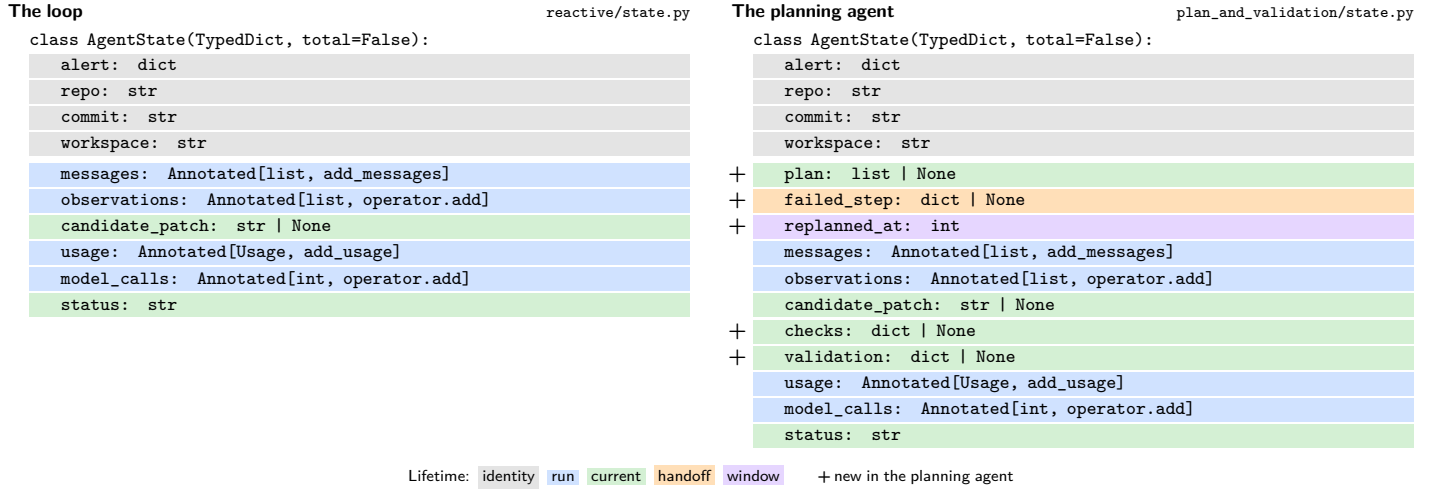


Figure 5: The two `AgentState` classes, with comments omitted. Colours show how long field values remain valid (Section 2.4). The + marks five fields added by the planning agent. Three hold a current value, as some fields in the loop already do. `failed_step` and `replanned_at` add the handoff and window lifetimes. Neither agent retries whole attempts, so neither schema has attempt-lifetime fields.

then meet the following conditions.

- **Admission** (Definition 6.1). Before a model call starts, the program checks $c < B$. Since the recorded count matches the actual count, completing one more call increases k by one without exceeding B .
- **Counting**. Every completed call adds exactly one to c . If a call is omitted, c falls below k , and later admission checks can permit too many calls. Counting a call twice uses up the recorded budget too early.
- **Preservation**. A step that makes no call leaves c unchanged. No step, including plan revision, decreases it. The count therefore needs the run lifetime described in Section 2.4. Resetting c after completed calls leaves $c \leq B$ true but makes $c = k$ false. Later checks can then allow the actual call count to exceed B .

This proof covers completed calls. An exception may interrupt a call after it has incurred a cost but before it is counted. The condition P does not account for that cost.

An accumulating reducer helps by adding each reported call to c . Each node must still check the budget before calling and report every completed call exactly once. **The reducer adds the reported counts; their accuracy depends on the callers.** In our agent, c is `model_calls` and B is forty.

2.6 State schemas of the loop and the planning agent

The Python class `AgentState` defines our schema as a `TypedDict` (Figure 5). Each key names a state field, and its type annotation specifies the kind of value it holds. Writing `Annotated[T, fn]` also assigns the reducer `fn`. The setting `total=False` makes every key optional for type checking, which allows the partial dictionaries returned by nodes. `TypedDict` supports static type checking [12]; it does not check values while the program runs.

When we construct `StateGraph(AgentState)`, `LangGraph` reads the schema and creates a channel for each field. A channel stores the value and applies its update rule: the specified reducer, or replacement by default. We then build the graph using these operations:

- `add_node(name, fn)` registers a node: a function that receives the whole state and returns a partial update.
- `add_edge(a, b)` fixes a transition from node `a` to node `b`.
- `add_conditional_edges(a, route, {...})` registers a routing function to run after `a`. It reads the updated state and returns the next node's name without writing state.

- `compile()` checks the graph's connections and returns a runnable graph. `invoke(initial_state(...))` starts the run with a value for every key, so nodes do not read missing keys.

```

graph = StateGraph(AgentState)
graph.add_node("plan", self.plan)
graph.add_node("tools", self.tools)          # ... and three more
graph.add_edge(START, "plan")
graph.add_conditional_edges("tools", self.after_tools,
                           {"controller": "controller", "plan": "plan"})
graph.add_edge("validate", END)
app = graph.compile()

```

The loop's schema (Figure 5, left) uses three lifetimes. Identity fields identify the scanner alert, test repository, fixed commit and workspace copy. Fields retained for the whole run include the histories and totals used by stall and budget checks. Updates append to `messages` and `observations`, and add to `model_calls` and `usage`. The current fields, `candidate_patch` and `status`, hold the latest patch and execution status.

Our `add_usage` reducer keeps four separate token totals. Cached input costs less than fresh input, so a combined total could hide an increase in one alongside a decrease in the other. In both agents, however, the code that reads model replies fills only the input and output counters. The two cache counters are zero in both recorded runs of the planning agent.

The planning agent (Figure 5, right) adds five fields. The plan node writes `plan`, the submit node writes `checks`, and the validator writes `validation`. Each holds the latest value, so these three fields use the current lifetime. The remaining two fields use handoff and window lifetimes.

- **Handoff**. When the tools node detects a stall, it writes the reason to `failed_step`. The routing function reads this field and selects the plan node. That node reads the reason, revises the plan and clears `failed_step` to `None`. If it left the field set, the next pass through `tools` would return to the plan node again.
- **Window**. The `observations` list keeps growing and is never cleared. Immediately after a plan revision, its last three entries are still the repeated results that caused the revision. To avoid detecting the same stall again, the plan node stores the current observation count in `replanned_at`. The stall check then reads only `observations[replanned_at:]`. A nonzero value also records that the one permitted revision has been used. If the run stalls again, it ends.

Each field passes information from a step that writes it to a later step that reads it (Table 3).

Fields	Why they persist and which steps read them
alert, repo, commit, workspace	Identify the task and its permitted workspace throughout the run. The workspace path names external state; the files are not in the record.
plan	The plan node writes it and adds a rendered message to the controller’s conversation. A revision reads the previous plan.
failed_step, replanned_at	The tools node records a stall; routing sends the run to the plan node, which clears failed_step and sets replanned_at to the index where observations under the revised plan begin. Stall checks read from that index onward.
messages	The opening instructions, plan messages, controller replies, tool replies and check feedback form the controller’s conversation. This field excludes the replies of the plan node and validator.
observations	The tools node appends one structured result per tool call; the stall checks and the validator read them.
candidate_patch, checks	The submit node stores the patch and the check results, whether passed or failed. Routing tests the checks, and validation reads both.
validation, status	The validator stores its verdict, its reasons, and whether the model or runtime decided. status also records runtime stop reasons. Routing and the code that runs the graph and reports its outcome read this status.
model_calls, usage	Each of the three model nodes adds one call and that call’s tokens. Admission reads the call count; only the run’s closing summary reads the tokens.

Table 3: The planning agent’s state fields, showing which nodes write them and which later steps read them.

The following code shows the difference between model context and runtime state (Section 2.2). The controller sends the conversation to the model. Before making that call, the runtime compares the call counter with the configured budget. **The runtime checks the budget before the call; the controller’s context does not contain it.** The planning agent’s other prompts also omit the budget.

```
# controller: the model receives the conversation
reply = self._call(model, state["messages"],
                  role="controller")

# admission: run before every model call
if state["model_calls"] >= self.config.budget:
    return {"status": TerminalStatus.BUDGET_EXHAUSTED}
```

We define a state graph in the following order.

1. Name the nodes and the routing decisions between them.
2. List the values that nodes and routing functions must read. These values define the schema. Omit fields that no computation reads.
3. For each field, identify which node writes it and when the value becomes outdated. Choose its lifetime using Section 2.4.
4. Choose a merge rule that supports that lifetime: no custom reducer for identity, append or accumulate for run, and replacement for current, handoff and window. For an attempt field, make sure the rule allows the value to be cleared.

5. Write the initial state so that every key has a value before the first node runs.
6. Have each node return only the fields it is responsible for updating. When adding a field, also add the node code that supplies its value.

2.7 Reading

LangChain [9], the implementation reference for state, schemas and reducers; and MIT 6.042 [17], §2 (p. 5), on invariants and the Invariant Theorem.

For reference while writing HW1: Lehtosalo [12] on what a **TypedDict** does and does not check at run time, and LangChain [10] on the checkpointers, threads and stores used for durability in Table 2.

Looking ahead. Once the runtime has applied an update, a routing function reads the resulting state and selects the next node or ends the run. We next examine the rules behind these choices, the loops that return to planning or the controller, and what a trace tells us about how a run ended.

3 Routing, feedback, and stopping

3.1 Routing functions and guards

The routing function uses values in the record to choose the next step. Model output supplies some of these values. The programmer writes the rules that the function applies to them.

Definition 3.1 (Conditional edge). A conditional edge consists of the transitions that leave a control location and a routing function that chooses among them. After the runtime merges the location’s update, the routing function reads the record. It tests the guards on the outgoing transitions and returns the destination selected by those tests.

In the notation of Section 1.3, a guard is a predicate $g(v, e)$: a true-or-false test on the variables and the step’s input. The routing function tests the merged record $v' = \text{merge}(v, u)$, which includes the information the node kept from e . **A guard can test stored information; it cannot test information the node discarded.** Routing is deterministic in the sense of Definition 1.2 when it selects exactly one destination for every merged record. One way to do this is to test conditions in order, take the first match, and use a default if none matches. The order decides which transition to take when several conditions hold. The default may still select an unsuitable destination for a record the programmer did not anticipate.

A graph also needs a destination that ends the run. LangGraph provides it as **END**: “The END Node is a special node that represents a terminal node” [9]. A routing function that returns it stops the run. The record states the reason.

The model, runtime, and routing function each have a role in this process.

- **Proposal.** The model emits a tool request, a plan, a candidate patch or a verdict. The runtime must process this proposal before it can take effect.
- **Record.** The runtime uses the reducers of Section 2.3 to merge the proposal into state. It also records what happened in the trace.
- **Decision.** The routing function reads fields of the merged record. It applies the programmer’s guards and limits to choose a destination.

In our planning agent, **after_controller** tests the status first. If the run is no longer active, it returns **END**. If the run is active, it checks the last reply for tool requests. It returns **tools** when it finds requests and **submit** when it finds none. The function does

not read the reply’s prose. A reply that says “the patch is correct, submit it” while requesting a tool therefore routes to `tools`.

A routing event records `by="model"` when the tested value came from model output. It records `by="runtime"` when that value came from a check, a limit, or a runtime default. **The field identifies the source of the tested value. It does not mean that the model chose the edge.** The programmer defined the edge and its guard.

3.2 The same graph, different paths

The compiled graph contains the locations, unconditional edges, and routing functions. Its structure stays the same across runs. The records produced during a run can differ, so the routing functions can choose different destinations.

Using the terms of Section 1.4, the compiled graph defines the machine. Each run follows one trajectory through its computation tree. Each visit to a location appears as a separate node in that tree, with its own configuration. A run that calls the controller twelve times passes through twelve branch points without changing the graph. **Compilation fixes the possible paths; the values produced during a run determine which path it takes.**

Figure 1 shows our planning agent’s machine, and Figure 2 unfolds part of its computation tree. Section 3.6 examines two recorded runs that follow different paths.

3.3 Feedback edges

A feedback edge returns control to a location the run has already visited. To define one, we must state what triggers the return and what information goes back with it. If the edge restarts work, we must also state which fields keep their values and which are reset.

Agents of this kind commonly use two feedback edges with different triggers and destinations.

- **Revision.** A runtime check detects a stall and stores the reason in a field for the plan node to read. The routing function returns control to that node. It reads the reason, replaces the plan, and clears the field. A marker identifies the first observation under the revised plan. Later stall checks use only observations from that point onward (Section 2.4).
- **Repair.** When a proposed artifact fails a mechanical check, the runtime adds the check’s reasons to the conversation. The routing function returns control to the location that proposed the artifact. No fields need resetting. The failed proposal stays in the conversation, and the next proposal replaces the one stored in the artifact field.

The revision edge responds to a problem with the plan. The repair edge responds to a problem with the artifact. Both allow the run to continue, so both need a limit on how often they can be taken. **A feedback edge without its own counter depends on the limits for the whole run.** Our planning agent has both edges. The first stall returns control to the plan node once. Failed checks return control and their reasons to the controller as often as the call budget allows.

The proposed extension adds a third feedback edge (Figure 6). In the current planning agent, the validator’s verdict always ends the run. In the extension, a rejection would lead to a reflection step that records what went wrong. A new attempt would then start at the plan node. We would need to define which fields this transition resets. The failed attempt’s plan, patch, and check results would mislead the next attempt if left in place. These fields have the attempt lifetime defined in Section 2.4, so the transition must clear them. The reflection, attempt counter a , and call counter have the run lifetime: they must not be cleared when an attempt restarts. **Resetting the attempt counter with each attempt prevents it from reaching its limit.**

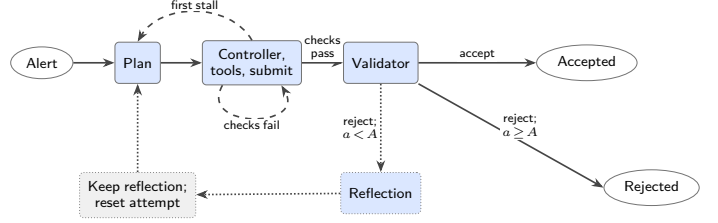


Figure 6: A proposed extension to the planning agent. Solid and dashed edges exist in the implementation; dotted nodes and edges show the extension. Blue boxes make model calls, the grey box performs runtime work, and ellipses mark the input and two endings. Dashed arrows indicate feedback. The middle box represents the controller, tools, and submit steps. a counts attempts and A is the attempt limit. In the implemented planning agent, every rejection ends the run.

3.4 Loop bounds and termination

An exit from a loop does not by itself guarantee termination. An acceptance predicate tells us when a run has succeeded, but the run may never satisfy it. If patches keep being rejected or are never submitted, another condition must stop the run. A termination argument shows that the run will eventually reach a condition that stops it.

Definition 3.2 (Variant). To show that a loop terminates: “Assuming the loop invariant holds at the start of each iteration, show that some quantity strictly decreases, and that it cannot decrease indefinitely. This quantity is called the decremting function or loop variant.” [5]

For a graph, we treat each cycle that continues the run as an iteration. Our variant is a nonnegative integer computed from the record. It must decrease on every such cycle. Because it cannot fall below zero, its initial value limits the number of cycles. This argument places no limit on how long an individual step can take.

The number of remaining calls, $B - c$, is one possible variant. The call invariant of Section 2.5 keeps $c \leq B$, so this number is nonnegative. To use it, we need two conditions: i) every cycle makes a model call, and ii) every step between calls returns or times out. A cycle that consists only of runtime work makes no model call. Examples include a check that repeats itself or a tool retry that does not consult the model. In these cycles, $B - c$ stays constant. **A cycle that consumes no model calls needs its own bound.**

A stall check looks for a particular pattern that indicates a lack of progress, such as repeated identical results. When it finds that pattern, it ends or redirects the run. A run can fail to progress without producing the pattern. For example, each search may return a different irrelevant result. **Detecting one failure pattern does not prove termination.** ReAct’s recorded trajectories include repeated actions that make no progress [20, §3.3].

Three runtime checks can end a run before a verdict. Each compares values in the merged record.

- **Budget.** The runtime checks the model-call limit before each call. By the time a call has finished, its cost can no longer be prevented. When each cycle uses this limited call budget, the number of remaining calls is the variant.
- **Stall.** The runtime checks for repeated results. The first stall can trigger a plan revision, and the next can end the run. This limits the number of revisions. It gives no bound on a run that never produces the repeated results.
- **Recursion limit.** LangGraph limits the number of super-steps (Definition 4.1) in one execution. It raises an error when execution reaches that limit [9]. This check uses no information about the task or its costs. It provides a fallback when errors in

Feedback edge	Trigger	Information passed back	What is reset	What bounds it
Revision: tools to plan node	Three identical observations within the current plan’s window, none a successful edit.	The stall reason and the previous plan, read by the plan node.	The plan is replaced, the handoff field cleared, and the start of the stall window moved; observations are kept.	At most once per run. A second stall ends the run with <code>no_progress</code> .
Repair: submit to controller	At least one of the three checks fails.	The checks’ reasons, as a message in the conversation.	Nothing. The next submission replaces the stored patch and check results.	The call budget alone.
New attempt: validator to reflection to plan node (proposed, not implemented)	The validator rejects, and fewer than <i>A</i> attempts have been made.	A written reflection on the rejected attempt.	Each field with an attempt lifetime is explicitly cleared.	The attempt limit, whose counter keeps its value across attempts, and the call budget.

Table 4: The planning agent’s two implemented feedback edges and the proposed third edge. Only the first two rows describe current behavior.

the graph’s connections would otherwise let execution continue.

In our planning agent, the call budget is forty calls, and the recursion limit is four times the call budget.

3.5 What a trace establishes

When reading a trace, we must distinguish three claims: i) execution terminated; ii) the acceptance procedure passed; iii) the artifact satisfies the task’s postcondition, the property required of the result. Each claim is stronger than the previous one. A terminal event establishes the first. If its status is accepted, it also establishes the second. No event establishes the third. The acceptance procedure provides evidence about the artifact, but it does not prove that the artifact meets the postcondition. **The trace records the outcome of acceptance checks. It does not establish that the patch is correct.**

The trace writer uses a fixed vocabulary of five terminal statuses: `accepted`, `rejected`, `budget_exhausted`, `no_progress` and `error`. It refuses other values. Closing a trace without a terminal event raises `NoTerminalStatus`. **If a trace closes normally without a terminal event, the trace recording has a defect. The missing event cannot serve as an outcome.** When an exception escapes the run, the writer records terminal status `error`, then writes a record naming the exception. The status identifies the kind of ending; the next record identifies its cause.

If the process receives `SIGKILL`, no handler runs. The file ends at the last written event, without a terminal or exception record. After a power loss, even that last event may be missing. The writer flushes events to the operating system, which may still be holding them in memory, but does not force them to disk (Table 5).

These interpretations depend on three distinctions.

- **Errors and outcomes.** A refused tool call is an observation inside a run that can still end `accepted`, while a process killed between two events leaves a run with no status at all.
- **Findings and status.** An assessment can finish with a report that the information is insufficient. A run can also stop with some evidence collected but no finding made.
- **Reaching the end and the reason.** Reaching `END` establishes only that the graph stopped. The reason is in the status, which the runtime sets.

The runtime controls termination as well as routing. A model’s claim that it has finished is a proposal. The runtime applies completion rules and sets the status; the trace records that status.

What the trace records	What it supports
A verdict, then a terminal event with status <code>accepted</code>	Execution terminated and the acceptance procedure passed. Whether the patch meets the task’s postcondition is a separate question the trace does not answer.
A terminal event with a stop status	Execution terminated without acceptance, for the stated reason, with whatever partial evidence the run had gathered. For error , the record that follows names the exception.
No terminal event	This record does not establish that the run ended. The run may still be active. Its process may also have stopped without running a handler, as would happen after <code>SIGKILL</code> or a power loss.

Table 5: Three interpretations of a trace. The absence of a terminal event limits what the record establishes. Calling that absence an `error` assigns a cause without evidence.

3.6 The demo: recorded endings and repairs

Our planning agent implements these routing rules and limits. After submission, `after_submit` chooses among three destinations. It returns `END` if the status has changed, `validate` if all three checks passed, and `controller` otherwise. The stall check looks for three consecutive identical observations, none of which is a successful edit. The first stall returns control to the plan node, which records `replanned_at`. The next stall check uses only observations since that point. If it detects a second stall, the run ends with `no_progress`. Neither recorded run revises its plan.

The number of remaining calls is a variant for the whole graph. Every cycle passes through the controller, which either makes a call or ends the run. The recursion limit cannot be reached first. Every step without a model call is followed by a step that makes a call or ends the run, so a run takes at most about twice the call budget in supersteps. We can only partly establish that steps return or time out, as required in Section 3.4. Rescanning and patch application have timeouts. For model calls and file tools, we depend on any limits imposed by the client library and operating system. Our configuration sets none.

The two recorded runs of the planning agent have different endings. One includes two repairs. The code also defines a third ending, budget exhaustion. The numbers below come from the `step` fields in the trace files.

- **Accepted.** Run bc284c6b ends **accepted** after one submission; [Section 1.5](#) walks through its events.
- **Returned for repair twice.** Run 28e37a00 submits at step 24. At step 25 the patch applies, but the edit does not touch the flagged line and the rescan still reports the finding. Step 26 routes back to **controller**, recording `by="runtime"` and the two reasons. The controller receives those reasons as a conversation message. After three tool rounds (steps 27–38), it submits again. At step 41 the rescan is clean, but the flagged-line check still fails. Step 42 routes back with that reason.
- **Interrupted.** The same run makes two more tool rounds (steps 43–50) and is then interrupted from the keyboard. Step 51 is a terminal record with status **error** and token totals of 67,586 input and 10,380 output; step 52, written after it, records the exception **KeyboardInterrupt**. The interrupt handler writes both records. The last event from the agent’s work is the tool result at step 50. The trace does not establish what the process was doing between that event and the interrupt.

A SIGKILL at the same moment would leave the file ending at step 50, a tool result, without terminal or exception records. No code would run to raise **NoTerminalStatus**. The first 50 records would be identical. **The two files would differ only in the records written after the work stopped.**

Neither recorded run ends with `budget_exhausted`, so we use the code to describe that ending. The configuration accepts smaller budgets, such as ten calls. A run that reached this budget would record a routing event with decision `budget_exhausted`, `by="runtime"`, and the call count and budget, followed by the terminal record. No model call would occur between them because the runtime checks whether it can admit the call before making it. The gathered evidence would remain in **observations**.

3.7 Reading

Read the sections on conditional edges, **END** and the recursion limit in LangChain [9], and the argument for loop correctness through the Termination step in Cornell CS 2112 [5]. §3.3 of Yao et al. [20] describes the repeated-action failure mode that stall checks detect.

When writing HW1, consult the terminal statuses and trace events in `trace.py`. These define the vocabulary used in [Table 5](#).

Looking ahead. After routing selects a location, the runtime schedules its computation. The next section examines scheduling and tool execution: what a tool contract specifies beyond the model’s schema, how the dispatcher processes a request, and which component refuses operations the run is not permitted to perform.

4 Scheduling, tools, and execution authority

4.1 Scheduling: when an update becomes visible

The routing function chooses the next location. The scheduler determines when the computation at that location runs, which version of the record it reads, and when later computations can read its update.

Definition 4.1 (Superstep). “Pregel computations consist of a sequence of iterations, called supersteps.” By design, the model “doesn’t expose any mechanism for detecting order of execution within a superstep, and all communication is from superstep S to superstep $S+1$ ” [15, §1].

A runtime using this model divides each superstep into three phases. i) During *planning*, it selects computations whose inputs were written in the previous superstep. ii) During *execution*, it runs those computations. Their writes remain invisible to other computations in the same phase. iii) During *update*, it applies the writes using the reducers of [Section 2.3](#). The next superstep’s planning

phase uses the updated record. In the notation of [Section 2.1](#), execution computes u_i , and the update phase computes $\text{merge}(s_i, u_i)$.

The runtime therefore applies a node’s writes after the node returns and before planning the next node. **A later node never reads a partly applied update.** Nodes in the same superstep cannot see one another’s writes. The runtime combines those writes in the update phase. If two nodes write the same field, that field needs a reducer to combine their values. Without one, the runtime must reject the competing writes.

This guarantee applies only to the record. A tool can change a file during execution, before the superstep ends. If the node then fails, the file remains changed, but the record receives no update describing that change. **The superstep rules determine when record updates become visible. They do not control changes to files or other external resources.** The checks around tool execution must control those effects.

4.2 The tool contract

The model receives a tool’s name, description, and argument schema. The runtime holds the code that implements the tool. It enforces restrictions on what the tool may do and handles invalid arguments. The description alone cannot enforce these restrictions.

Definition 4.2 (Tool contract). A *tool contract* specifies five parts of a tool: i) its name and purpose, ii) the types of its arguments and the restrictions on their values, iii) the type of a successful result and how errors are represented, iv) its side-effect class, which describes the kinds of external changes it can make, and v) restrictions on execution, such as the directories it may access. The definition supplied to the model covers the first two parts. The tool’s implementation and the dispatcher determine the other three. The model cannot establish them from the description.

A method specification states what the caller and the implementer must each ensure ([16](#), Specification structure).

Definition 4.3 (Precondition). “The precondition is an obligation on the client (the caller of the method). It is a condition over the state in which the method is invoked.”

Definition 4.4 (Postcondition). “The postcondition is an obligation on the implementer of the method.” “In general, the postcondition is a condition on the state of the program after the method is invoked, assuming the precondition was true before.”

For a tool, the precondition states which argument types and values are allowed, including any restrictions on execution. For example, a path may have to resolve inside the workspace. The postcondition states what the tool promises about its result, errors, and side effects. These promises apply when the precondition holds.

A method specification may make no promises if the caller violates a precondition. An agent’s tool contract needs stricter rules for this case, because the model can supply invalid arguments during normal operation. The contract must state which violations the runtime detects and how it reports a refusal. It must also require that a refused request has no side effects. **A tool contract must define how the runtime handles violated preconditions.**

Our three tools illustrate this contract. `read_file` takes a path and an optional line range, then returns text. `search` takes a regular expression and an optional glob, then returns matching lines with their locations. `edit` replaces one exact occurrence of

a string in a file. Only `edit` writes. All three take the workspace root as their first parameter. Before opening a file, `read_file` and `edit` resolve its path using that root. `search` matches its glob against the root without resolving paths (Section 4.6).

The dispatcher uses the function signature to decide which arguments to accept. This also determines who can supply configuration values. Our tools read the run’s configuration from the module; they have no configuration parameter. The dispatcher binds the model’s arguments to the parameters in the signature. As a result, **a parameter the dispatcher has not already supplied can receive a value from the model**. Adding a `config` parameter would let the model set it. The dispatcher itself supplies the workspace-root parameter, which is absent from the model’s schema. If the model names that parameter anyway, binding fails because the dispatcher has already supplied its value.

4.3 The dispatch cycle

The dispatch cycle processes a tool request in five operations. Each operation can end the cycle. Figure 7 follows one request through them.

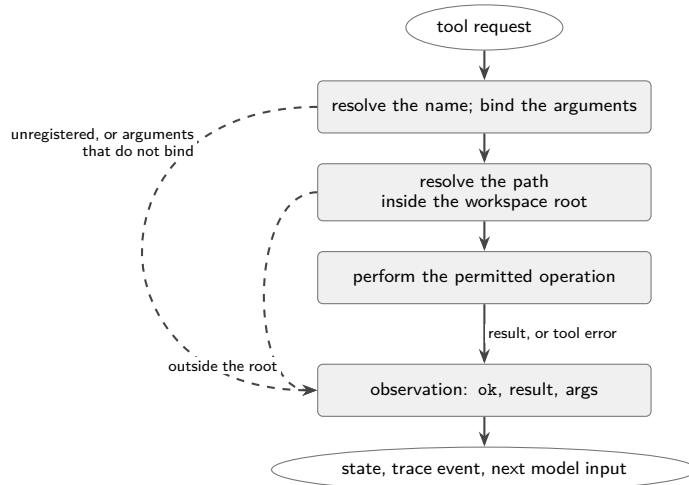


Figure 7: The stages of processing one tool request. Ellipses show the incoming request and the records that receive the observation; grey boxes show runtime work. Solid arrows follow successful checks. Dashed arrows show refusals, which also produce observations. A policy check may run inside the tool, as in our implementation, provided it runs before the operation it protects.

- **Resolve the name.** The dispatcher looks up the requested name in its registry, which it builds from the tool enumeration. If the name is absent, it refuses the request and lists the available tools in its response. Supplying a tool definition to the model cannot make an unregistered tool run.
- **Bind the arguments.** Before calling the function, the dispatcher matches the model’s arguments to its signature. If an argument name is wrong, it reports a readable refusal without letting a `TypeError` arise inside the tool. Binding checks argument names and arity, the number of arguments. It does not check their types. The dispatcher separately converts values when the required conversion is unambiguous. For example, it can convert a string supplied where an integer was required, avoiding an extra model call to correct the request.
- **Check the policy.** Before opening a file, the tool resolves its path. If the resolved path lies outside the workspace root, it refuses the request. This check is the first operation in `read_file` and `edit`. The dispatcher converts any exception from the check into a refusal.
- **Perform the permitted operation.** The function runs with the bound arguments and the dispatcher’s workspace root.
- **Observe.** The dispatcher creates an `Observation` for both

success and failure. It contains the tool, the arguments, an `ok` flag, and a string result. The dispatcher also writes a `tool_result` event in either case.

A refusal returns an observation to the model, allowing the reactive loop to continue. An exception that escapes this handling ends the run. If `edit` refuses an ambiguous target, the model can use the observation to retry with more context. Allowing an exception to escape would end the run at that first imprecise edit. Exceptions that end the run are reserved for unrecoverable failures, such as an unwritable trace.

If a model response contains several tool calls, the tools node dispatches them one at a time within one superstep. It returns an observation for each call, including failed calls. If it omits a response to any call, the next model message is malformed.

4.4 Evidence of a request and its execution

We can examine one tool call in six places. Each supports a different claim about what happened. Table 6 follows the first `read_file` call of the planning agent’s accepted run, `bc284c6b`, through these records and processing stages.

Artifact	What it contains	What it establishes
Model input	The tool definitions and the conversation so far. The trace does not record this input.	Which operations and descriptions the model received.
Model response, and the <code>model_call</code> event (step 3)	A tool call with an identifier, a name and arguments. The event keeps only the name, <code>read_file</code> .	That the model requested an operation. It does not establish execution.
<code>tool_request</code> event (step 5)	The tool name and the arguments as the dispatcher received them.	That the request reached the dispatcher and began the cycle.
Dispatcher	Registry lookup, argument binding, and path resolution. These have no separate event. If a check failed, the result event would give the reason.	That the request satisfied the contract, or why it failed a check.
<code>tool_result</code> event (step 6)	<code>ok=true</code> , 1,665 characters, which is the length of the file.	That the function ran and returned, and how much text came back.
State and next input	One record added to <code>observations</code> , one tool message added to <code>messages</code> .	What the next model call, at step 7, was allowed to see.

Table 6: Records and processing stages for one `read_file` call on `testcode/BenchmarkTest00283.py`, from run `bc284c6b`, steps 3 to 7. Step numbers count trace events rather than supersteps. The message history retains the provider’s raw request; the trace records the request received by the dispatcher.

Step 4 falls between the model response and the tool request. It is a routing event with decision `tools` and `by="model"`, because the reply contained a tool call (Section 3.1). It records the selected control location. It does not describe tool execution.

The model response tells us what the model requested. It cannot establish that a file operation occurred. Even if the model writes both a request and a plausible result in a transcript, neither shows that the tool ran. In this example, the runtime’s `tool_result` event has `ok=true`, establishing that the function ran and returned.

We must check this flag, because a result event can also report a refusal before the function is called. The successful event records the length of the returned text. It cannot establish the text’s contents.

4.5 Execution authority

The model proposes actions; the runtime executes or refuses them. We can identify the component responsible for each step as the request is processed. We must also show that the required checks run on *every* route by which an action can execute.

Definition 4.5 (Complete mediation). “To attempt to secure this resource, server needs to check all accesses to the resource. ‘Complete mediation.’ ” “Server will put a ‘guard’ in place to mediate every request for this particular resource. Only way to access the resource is to use the guard.” [8, §5]

For an agent, a resource is anything a tool can affect, including files, processes, or the network. The request is a tool call. The guard consists of the checks that run before the tool affects the resource. Complete mediation means that every route to the resource passes through those checks. Testing one route cannot establish what happens on a route that bypasses the checks. We therefore need to list every route to each protected operation. It is easy to overlook a route, such as a new tool or an argument that identifies a file without being named a path. Complete mediation ensures that the runtime checks the policy on every route. Whether the policy itself is correct is a separate question.

The term “control” can refer to two different questions.

- **Who decides, and when?** We fix some choices when we write the program. The program computes others while it runs. The time of a decision does not tell us who controls it: a routing function and a limit check both run during execution, and both apply rules the programmer wrote.
- **Who proposes, and who executes?** A requested action, a claimed result, or a declaration that the work is finished takes effect only when the runtime processes it. A model’s statement that it ran the tests does not establish that tests executed.

Before comparing the workspace root and the requested path, `workspace.resolve` fully resolves both. It follows symlinks and resolves `..` segments, so neither can hide a destination outside the root. Checking only whether one string starts with another would fail: `/tmp/ws` is also a prefix of the unrelated directory `/tmp/ws-evil`. Joining an absolute path to the root yields the absolute path itself. The check refuses it if its resolved destination lies outside the root.

4.6 The demo: scheduling and request refusal

Our planning agent runs on LangGraph’s runtime. It organizes execution “into multiple steps, following the Pregel Algorithm/Bulk Synchronous Parallel model”. During execution, “channel updates are invisible to actors until the next step” [11]. Our graph has no parallel branches, so each superstep runs one node. In run `bc284c6b`, steps 3 and 4 belong to the controller’s superstep: first the model call, then the routing event. The routing function reads the record after the controller’s reply has been merged. Steps 5 and 6 belong to the next superstep. The tools node reads the reply and dispatches its call, and the dispatcher writes both trace events during execution. The runtime adds the observation to **observations** in the update phase, after the node returns.

No recorded run contains a refused request. For this demonstration, we send requests directly to the dispatcher, outside a graph. We use a fresh workspace for the worked alert and report the dispatcher’s responses. The first request calls `read_file` on `testcode/BenchmarkTest00283.py`. The dispatcher permits it and returns the same 1,665 characters as step 6 of the recorded

Guarantee	Where it is enforced
Only registered tools run	The dispatcher checks the registry, built from the tool enumeration, before calling a tool.
Arguments bind to the tool’s signature	The dispatcher binds arguments and converts values before the call.
No file outside the workspace is written	<code>workspace.resolve</code> , called by <code>edit</code> , the one tool that writes, on its path argument.
No file outside the workspace is read	Not enforced. <code>read_file</code> resolves its path, but the glob of <code>search</code> never reaches <code>workspace.resolve</code> .
The pinned fixture is not modified	Every tool works on a disposable copy of the fixture, and the diff compares that copy with the pinned fixture.
An exhausted budget prevents another model call	The check each model node makes before it issues its call.
A plan is revised at most once per run	The tools node records a stall for revision only while <code>replanned_at</code> is unset. The routing function after tools reads that record (Table 4).

Table 7: Desired guarantees and the components that enforce them. The implementation does not enforce one of these guarantees. Prompt instructions enforce none of them.

run. The second request changes only the argument.

- **The request.** `{"name": "read_file", "args": {"path": "../etc/passwd"}}`, which no model produced.
- **The check that refuses it.** The first operation in `read_file` is `workspace.resolve`. It raises `PathEscapeException`, which the dispatcher catches.
- **The record.** The dispatcher returns an observation with `ok=false` and the result `refused: '../etc/passwd' resolves outside the workspace root`. It also writes a `tool_result` event with the same reason.
- **Evidence that no file was opened.** The trace contains a `tool_request` event and a failed `tool_result` event with a reason and no character count. These events show that the runtime refused the request. To establish that no file was opened, we must also read the code: the check runs before the first file operation. The trace records no file operations, so it cannot establish this on its own.

The two requests use the same tool and dispatcher. Their arguments determine whether the runtime permits or refuses them. This example identifies the component that enforces the rule described in Section 4.5.

The same exercise shows a route that bypasses the check. The dispatcher permits a `search` whose glob reaches outside the workspace. With the glob `../*-withheld/testcode/*.py`, it returns lines from the withheld benchmark cases stored in a sibling directory. With enough `../` segments, it returns the matching line of `/etc/passwd`. Both results have `ok=true`. **The path check guards `read_file` and `edit`; it does not guard `search`, whose glob never passes through `workspace.resolve`.** The check covers every way these tools can write outside the workspace, because `edit` is the only tool that writes. It leaves a way to read outside the workspace unchecked. To close this route, `search` must pass every matching file through `workspace.resolve` before reading it.

4.7 Reading

Read §1 and §2 of Malewicz et al. [15] for supersteps and the rule that communication crosses superstep boundaries in one direction only. Read the section “Specification structure” of MIT 6.031 [16] for preconditions and postconditions, and §5 and §6 of LaCurts [8] for the guard model and ways to bypass complete mediation.

When writing HW1, consult LangChain [11] for the planning, execution and update phases in our runtime. Use Anthropic [3] for the exact format of requests and results that a provider sends and expects. See `dispatch.py` and `workspace.py` for the checks that run between a request and its result.

Looking ahead. In the execution model described here, a routing function selects a node and the scheduler runs it in the next superstep. The next section keeps the same machine and changes this scheduling mechanism: a computation emits an event, which the runtime dispatches to a registered handler. We then examine which component invokes the handler and when. The tool contract and dispatcher remain the same. We will need evidence that each transition occurs once under the new scheduling mechanism.

5 An event-triggered implementation

5.1 Events, handlers and inversion of control

A graph and an event loop can run the same machine. They differ in how they call the application code. In the graph, a routing function selects the next node, and the scheduler runs it in the next superstep (Section 4.1). In an event loop, a computation emits an event, and the runtime calls the code registered to handle that event. The following four definitions describe this arrangement. All four come from the same source [4, Event Handling; Understanding the Event Loop].

Definition 5.1 (Event). “An event is any occurrence that could necessitate a change in the program’s state.”

Definition 5.2 (Handler). Handlers are “pieces of code that understand how to respond to a particular event”; for events to reach them, they “must register with the event loop.”

Definition 5.3 (Event loop). “An event loop is a piece of code that repeatedly checks whether a new event has occurred and dispatches any detected events to code that can handle them.”

Definition 5.4 (Inversion of control). “Inversion of control is used to describe the paradigm where we write custom subroutines but allow an external entity, such as a framework, to control when these subroutines are executed.”

For an agent, an event can represent something that happened, such as *the plan is ready* or *the checks failed*. It can also represent a request, such as *run these tools*. The program represents both kinds of event as values. A tool request event asks a handler to do work. The request alone provides no evidence that the handler ran. The same distinction applies to a model’s tool request in Section 4.4: a request to read a file provides no evidence that the read occurred.

Inversion of control separates two responsibilities. Application code defines how each handler responds to its event: what work it does, which fields of the record it writes, and which event it emits when it finishes. The framework determines which handler runs, when it runs, and the order in which it processes pending events. **Application code defines the response to an event;**

the framework controls when the handler runs. The graph implementation divides these responsibilities in a similar way: we write node bodies and routing functions, and the scheduler runs them. The two implementations choose the next computation in different places. A graph uses a routing function on an edge. An event handler chooses the type of event to emit, and the registrations associate that type with a handler.

The registrations determine which event types have handlers. A loop that does not check its registrations can remove an event from the queue even though no handler accepts that event’s type. No handler then runs, although no component has decided to stop. The framework’s implementation determines whether it reports this as an error. We classify this case as a defect.

5.2 An event-triggered version of the machine

Changing the implementation changes when the runtime takes a transition while preserving the set of transitions. A state-machine diagram alone does not specify when a reaction occurs. In an event-triggered model, a reaction occurs when an input is present. In a time-triggered model, it occurs on the tick of an external clock [18, slides 23–25]. Either model can use the locations, guards and set actions of the planning agent’s machine (Figure 1). Our graph implementation uses no clock: each superstep begins when the preceding one finishes. The event implementation reacts when an event is present, so it is event-triggered under this definition.

The choice of trigger can still make a transition unreachable. A guard requiring every triggering input to be *absent* cannot fire when a reaction requires a triggering input to be present. Each transition of the planning agent becomes enabled when the preceding computation completes, so it can wait for an event reporting that completion. A transition such as *no reply arrived within thirty seconds* needs a timer to emit an event when the thirty seconds end. Without that timer, the runtime never takes the transition even though it appears in the diagram.

The event implementation needs a larger configuration to describe its execution. In the graph implementation, a configuration is (q, v) : a control location q and a valuation v , which assigns a current value to each variable (Definition 1.4). In the event implementation, the configuration is a triple

$$(E, \mathcal{H}, v),$$

where E is the sequence of pending events, $\mathcal{H}$ is the set of running handlers, and v is the record. The runtime maintains the event queue E and the running handlers $\mathcal{H}$ separately from the record, which stores the steps’ data. In a serial implementation, at most one event is pending or being handled. During active execution, either one event is pending and no handler runs, or one handler runs and no event is pending. We can then recover the location q by identifying the handler registered for the pending event or the handler already running. An implementation that permits two handlers to run at once can reach a configuration with no single location q . The machine in Figure 1 cannot describe such a configuration. Our demo’s validation step provides an example (Section 5.4).

Four rules preserve the machine’s behaviour when an event loop implements it.

- **Apply once, then enable.** Merge a handler’s contribution into the record exactly once. Complete this update before making its emitted event available for dispatch. The graph’s superstep model provides this ordering (Definition 4.1). An event loop provides it only if the handler writes before it emits and the runtime delivers the event only once. If a handler emits first, the next handler can read the record before the update appears. If the runtime delivers an event twice, the handler appends the same observations twice.
- **Distinguish idle from success.** An empty queue with no running handler means that nothing is scheduled. This

alone does not explain how the run ended or whether it ended intentionally. Establishing termination requires a terminal event carrying a status, just as it requires a terminal record in the trace (Section 3.5).

- **Stop dispatch at termination.** After dispatching the terminal event, the runtime must start no further handler, even if an event remains pending. In a serial implementation, no event can be pending at that point. An implementation with concurrent handlers must discard or refuse the remaining events.
- **Separate triggering events from log events.** A triggering event causes the dispatcher to run a handler. A log event records something that happened and starts no computation. For example, an event requesting tools starts the tools handler. A trace record of the same request documents that the handler was called; it starts no further computation.

The third and fourth rules both apply when a run ends. The terminal event ends dispatch, and the terminal record in the trace stores the status. If the process fails before writing that record, dispatch can end without a recorded status. The interpretations in Table 5 still apply.

5.3 Two frameworks

Two frameworks provide event dispatch through different APIs.

- **LlamaIndex Workflows.** The framework selects a step by event type. A step is a method decorated with `@step`, and “A step receives an event, does some work, and returns another event. That returned event triggers the next step whose type annotation accepts it” [13]. Users define event types as Pydantic classes. The framework also supplies two event types: **StartEvent**, whose fields are the keyword arguments passed to `run()`, and **StopEvent**, whose return ends the workflow with the value passed as its `result`. Each step’s signature declares the event types it accepts and emits. Before a run, the framework checks the event graph described by these signatures. Among other conditions, it checks that the graph contains a start event and a stop event and that every emitted event has a step that accepts it. The workflow’s **Context** includes a store for state that belongs to the run. Steps read and write this state through `ctx.store`. They use `edit_state()` when an update must finish before another step can update the same store [14]. Our demo uses this framework (Section 5.4).
- **CrewAI Flows.** The framework selects methods using completion events and route labels. A method decorated with `@start()` is an entry point. A method decorated with `@listen(m)` runs when method `m` completes and receives its output. A method decorated with `@router(m)` runs after `m` and returns a string label. Methods decorated with `@listen("label")` run when the router returns that label. The operators `or_` and `and_` combine triggers. With `or_`, the listener runs when any of the specified methods produces output; with `and_`, it waits for all of them. The flow’s `state` attribute holds shared state as a dictionary or a Pydantic model. There is no stop event: “the final output is determined by the last method that completes” [6]. To translate our implementation to CrewAI, we would turn each event type into a router label. Our code would also have to write the outcome σ to `state`, because the end of the flow does not carry that outcome.

The frameworks therefore distinguish idle from success in different ways. LlamaIndex provides an explicit terminal event. Its checks before execution inspect the event graph declared by step signatures for start and stop events and missing connections. Passing these checks does not establish that a particular run will reach **StopEvent**. CrewAI defines no terminal event. Our reading of its documentation is that a flow ends when no method remains to run, corresponding to an empty queue. The application must therefore record the outcome before the last method completes.

Using either framework leaves a further question: does the event implementation preserve the machine’s behaviour? To compare it with the graph implementation, we need a controlled design. In this design, we compare the implementations after each computation finishes and its contribution has been applied to the record. These points let us compare the effects of completed computations without depending on a handler’s internal steps or their wall-clock timing. We script the model’s replies and control the tool results so that both implementations receive the same inputs in the same order. A faithful translation preserves four properties at each point: i) the observations appended to the record, ii) the effects on the workspace, iii) the results of the acceptance checks, and iv) the terminal status and its meaning. This comparison is designed to isolate the scheduling mechanism: a graph superstep or an event dispatch.

If the implementations agree under these scripted inputs, we can conclude that they follow the same transition rules for those replies. This does not establish that they behave alike with a live model. The model’s replies depend on the messages each implementation constructs. Changing how a handler formats a message changes the model’s input even when the transition rules stay the same. This comparison also excludes concurrent execution.

5.4 The demo: the planning agent as a LlamaIndex workflow

The event implementation, `cyberbird/event_driven`, runs the planning agent’s machine as a LlamaIndex workflow. It preserves the machine’s locations, guards and limits. Each alert has a budget of forty model calls, checked before each call. Three identical results count as a stall, and the agent may revise its plan once. The workflow stores the run’s data in its context. **RunState** holds bookkeeping for the whole run. Each alert has an **AlertState** that holds its plan, messages, observations, calls and usage. Two steps are still stubs: `submit` runs no mechanical checks, and each validator accepts after a fixed delay without calling a model. No run of this agent has been recorded.

The six rows below the horizontal rule in Table 8 correspond to the machine’s locations. Step signatures define the connections between steps. The framework sends each event to the step whose signature accepts its type, so each step can run without naming the next step. The graph’s routing functions evaluated the guards. Here, the steps evaluate those guards to choose which event to return.

- **Admission.** Before calling the model, `plan` and `controller` reserve one call from the alert’s budget. If the budget is exhausted, the step returns **AlertDone** with status `budget_exhausted`. It makes no model call in that case.
- **Acting.** The controller returns **ToolsRequested** when the model’s reply contains tool calls, and **Submitted** when it contains none.
- **Revising.** While the agent has not stalled, `tools` returns **Observed**. At the first stall, it returns **Stalled**, which starts the plan step. That step writes a new plan and updates `replanned_at`. These updates provide the handoff and the start of the observation window described in Section 2.4. At a second stall, the tools step returns **AlertDone** with status `no_progress`.

The event implementation also adds three behaviours to the graph implementation. First, one run can handle many alerts. `scan` runs Bandit once and emits one **AlertFound** for each alert. The `gate` step queues the alerts it receives and starts one at a time. It starts the next alert only after receiving **AlertDone** for the current alert. Every event about an alert carries an `alert_id` that links the event to that alert’s entry in the store.

Second, validators run concurrently, and a judge collects their

Events the step accepts	Step	Events the step emits	Decided by
RunRequested	start	ScanRequested	runtime
ScanRequested	scan	one AlertFound per alert, or StopEvent	runtime
AlertFound, AlertDone	gate	AlertStarted, one alert at a time, or StopEvent	runtime
AlertStarted, Stalled	plan	PlanCreated, AlertDone	model
PlanCreated, Observed, ChecksFailed	controller	ToolsRequested, Submitted, AlertDone	model
ToolsRequested	tools	Observed, Stalled, AlertDone	runtime
Submitted	submit	ChecksFailed, or $V \times \text{ValidateRequested}$	runtime
ValidateRequested	validate	Verdict	model
$V \times \text{Verdict}$	judge	AlertDone	runtime

Table 8: The event implementation’s steps, as listed in the connection table in `workflow.py`. Each step’s signature names the events it accepts, and its return annotation names the events it emits. The rows below the horizontal rule implement the machine in Figure 1; the rows above it are additions. V is the number of validators, three by default.

results. `submit` sends one `ValidateRequested` for each aspect of validation: correctness, behaviour and scope. It sends these events through `ctx.send_event`, because a step can return only one event. The validate step has up to three workers, allowing the three validators to run at the same time. This gives the configuration described in Section 5.2, with several running handlers and no single location. `judge` stores each `Verdict` until all V verdicts have arrived, in any order. It accepts the alert only if every validator accepted. The judge collects verdicts without grouping them by alert. This is safe only because the gate admits one alert at a time. **The gate processes alerts one at a time and allows validators for the current alert to run concurrently.**

Third, the implementation distinguishes the end of an alert from the end of the whole run. An alert ends with an `AlertDone` carrying one of four statuses: `accepted`, `rejected`, `no_progress` or `budget_exhausted`. The gate receives this event and records the status. When it has a result for every alert found by the scan, the gate returns a `StopEvent` carrying all the results and ends the run. If the scan finds no alerts, it ends the run immediately with an empty result.

The implementation handles the four rules of Section 5.2 as follows.

- **Apply once, then enable.** Every write uses `ctx.store.edit_state()`, which locks the store. Each step writes its data before emitting the event that allows another step to read them. The budget check and reservation use the same lock. Two steps requesting admission at the same time therefore cannot both reserve the last available call.
- **Distinguish idle from success.** A `StopEvent` explicitly ends the run. The gate emits it after collecting all alert results; the scan emits it if it finds no alerts. Before the first run, the framework also checks that every emitted event has a step that accepts it. For this check, `submit` lists `ValidateRequested` in its return annotation even though it sends those events through `ctx.send_event`.
- **Stop dispatch at termination.** The gate emits `StopEvent` only when the number of results equals the number of alerts. Every alert has therefore finished when the gate ends the run.
- **Separate triggering events from log events.** This agent’s events trigger steps. The agent currently writes no trace of their execution. Our graph agents also write a trace (Section 3.5); the event implementation still needs this feature.

5.5 Reading

Read Cornell CS 2110 [4], the sections Event Handling and Understanding the Event Loop, for events, handlers, the event loop and inversion of control. Read Seshia [18], slides 23–25, for event-triggered and time-triggered reactions.

While writing HW1, refer to LlamaIndex [13], the sections on

entry points, exit points and validation, and to LlamaIndex [14] for the per-run store. For CrewAI, refer to CrewAI [6], the sections on `@start()`, `@listen()`, retrieving the final output, conditional logic and the router.

Looking ahead. In an event loop, the runtime calls a registered handler when its event arrives. That handler can emit any number of events. Deciding whether this work is permitted and measuring its cost require further mechanisms. The next section introduces admission, which decides whether a model or tool call may start, and accounting, which records its cost after execution. It also considers how to compare the costs of implementations under controlled conditions like those used here to compare behaviour.

6 Budgets and controlled measurement

6.1 Admission and accounting

To limit spending, a runtime needs two mechanisms that act at different times. Before a model or tool call, it must decide whether the call may start. After the call runs, it must record the cost. The first mechanism uses information available before the call. The second uses the usage reported afterwards.

Definition 6.1 (Admission). Admission decides whether a unit of work may start. It checks how much of the run’s limit is already committed, then permits or refuses the work. Refused work never runs and incurs no charge.

Definition 6.2 (Accounting). Accounting records the cost of a unit of work after it runs. It adds the charge to a running total, measured in the same units as the limit. Admission reads this total.

The two mechanisms work together to enforce the limit. If accounting leaves a call out of the total, admission can allow more work than the limit permits. If admission never checks the total, accounting can record an overrun only after it happens. **Accounting maintains the usage total; admission uses it to enforce the limit.**

Accounting reads the charge reported with each reply. Admission must decide before the reply exists, when it knows the input to be sent but has not seen the output. A call-count limit is therefore easier to enforce than a token limit. Each call counts as one, so admission knows its cost in advance. The token cost becomes known only after the call runs.

The graph implementation of our planning agent uses call-count admission. It checks a budget of forty calls before each model call and records the call’s input and output tokens. The token admission and reservation mechanism developed below is a

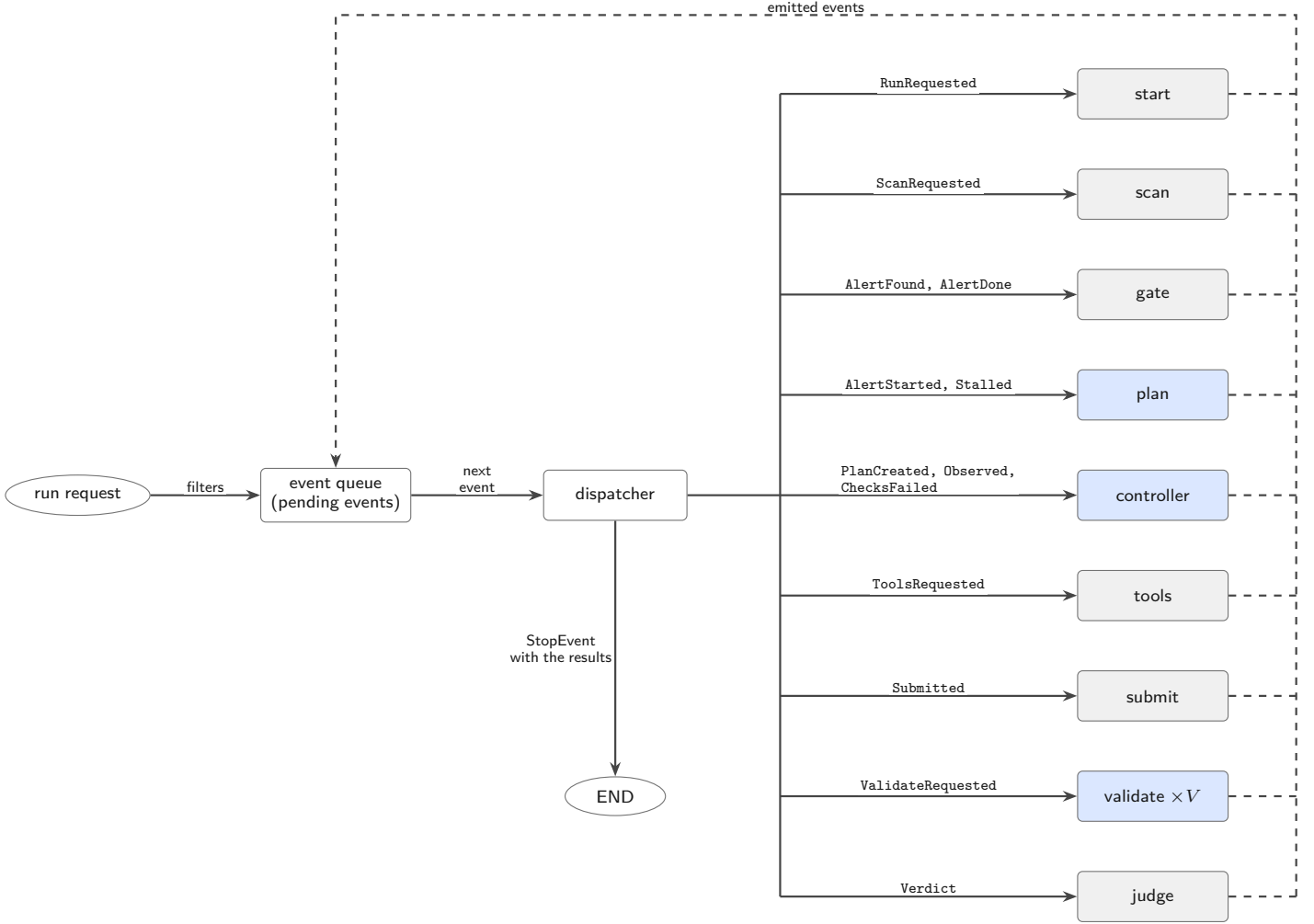


Figure 8: The event implementation of Table 8. White boxes are runtime components. Steps use the colours of Figure 1: blue steps call a model (the validators will do so once their stubs are replaced), and grey steps run tools, checks or bookkeeping. The ellipses mark the request that starts the run and END. Each solid arrow from the dispatcher names the event types accepted by the step it leads to. Each event emitted by a step returns to the queue along the dashed path. V validators run concurrently.

proposed design. Our code does not implement it.

Call counts provide only a rough measure of token usage. In our two recorded runs, individual calls have inputs ranging from 571 to 13,550 tokens. Each controller call resends the conversation, which grows during the run. A budget of forty calls limits the number of replies. The token cost of those forty calls still depends on the largest possible call.

6.2 A budget invariant

Admission and accounting must use a clearly defined budget unit. The choice of unit determines which cost the budget limits. We consider three choices.

- **Output tokens.** Count only tokens generated by the model. A per-call output limit sets a bound on each call’s cost before it starts, which simplifies admission. This budget leaves out input tokens, whose count grows as calls resend the conversation.
- **All processed tokens.** Count input and output tokens together to track the provider’s work. Admission then needs a bound on both the input and the output of each call.
- **Money.** Multiply token counts by their prices. Different kinds of tokens, such as cache reads and fresh input, can have different prices. Accounting must keep these counts separate until it calculates the charge. A change in prices changes how much work the budget permits.

The following argument applies to all three choices. The chosen unit determines how we calculate a call’s charge γ .

We state the budget rule as an invariant using three quantities. B is the budget. S is *settled spend*: the sum of recorded charges for completed calls. R is the total still reserved for calls that have been admitted but whose charges have not yet been recorded. These are the *outstanding reservations*. The invariant is

$$S + R \leq B.$$

A *reservation* ρ sets aside part of the budget for a call before it starts. This amount must be a justified upper bound on the call’s cost. After the call completes, *settlement* replaces its reservation with the recorded charge γ .

We want to bound the charges themselves: the total charge for every call the run has started, including unfinished calls, must never exceed B . As with the call counter in Section 2.5, the runtime can compare only the values it has recorded. We need a further requirement to connect the invariant over these recorded values to the charges:

$$\forall i. \gamma_i \leq \rho_i,$$

where i ranges over admitted calls, ρ_i is the reservation for call i , and γ_i is its charge on completion. Admission must choose ρ so that this requirement holds. The requirement is not a state predicate that the runtime can check: for an outstanding call, the runtime does not yet know the final charge γ_i . The input

and output bounds below justify the requirement. If it holds, $S + R$ bounds the total charge of all calls started. Keeping $S + R$ within B then keeps those charges within the budget. We use the invariant method (Definition 2.3) to show that the inequality holds initially and remains true after every transition.

- **Initially.** The run has admitted no calls and recorded no charges, so $S = 0$ and $R = 0$. The invariant therefore holds for any $B \geq 0$.
- **Admit.** Admission checks whether the new reservation fits: $S + R + \rho \leq B$. If it does, admission sets R to $R + \rho$, reserving ρ for the call. This guard ensures that the invariant still holds. If the reservation does not fit, admission refuses the call and leaves the record unchanged.
- **Settle.** The call completes with charge γ . Settlement adds this charge to the recorded total, setting S to $S + \gamma$. It also removes the reservation, setting R to $R - \rho$. Their sum changes by $\gamma - \rho$. Since $\gamma \leq \rho$, the sum cannot grow, and $S + R \leq B$ still holds. The unused amount $\rho - \gamma$ becomes available for later calls.
- **Release.** An admitted call may never be issued, for example if the run ends first. The runtime then removes the reservation by reducing R by ρ . The call incurred no charge, so the invariant continues to hold.
- **Every other transition.** Leave S and R unchanged. No transition decreases S : settled spend lasts for the whole run, as defined in Section 2.4. Resetting S to zero would make the budget available again even though the run had already incurred those charges.

Concurrent calls require one further condition: checking the guard and recording the reservation must form a single step. Suppose two model calls are admitted in the same superstep (Section 4.1). Both could check $S + R + \rho \leq B$ before either adds its reservation to R . Each check could then pass even though the combined cost exceeds B .

The proof depends on showing that $\gamma \leq \rho$. If the budget counts all processed tokens, ρ must cover both input and output. We need to justify a bound for each.

- **Input.** Establish an upper bound on the call’s input tokens. Before sending the call, the runtime can use the provider’s counting endpoint to obtain an estimate: “The token count is an estimate. In some cases, the actual number of input tokens used when creating a message might differ by a small amount” [2]. Comparing past estimates with recorded charges can help choose a margin to add to the estimate. A strict guarantee requires a justified upper bound on the estimation error. Another option is to use a limit that the request cannot exceed, such as the model’s context window.
- **Output.** Use an upper bound that the provider enforces. The request specifies a per-call output limit, and the provider stops generation when that limit is reached [1]. The bound holds even if the model would otherwise have generated more content.

An estimate alone cannot justify the invariant. If admission bases ρ on an estimate and the call costs more, settlement increases the sum by $\gamma - \rho > 0$. The record contains no flag showing which step caused $S + R$ to exceed B . **The guarantee from reservations requires an upper bound on each call’s charge.** Estimates help forecast spending. To guarantee that spending stays within B , we need the bounds described above.

The runtime must also handle unused reservations and uncertain charges. It must release unused reservations so that useful work can continue. This is a liveness requirement: an unused reservation keeps R high and can cause admission to refuse work that would otherwise fit. Keeping that reservation does not violate the spending invariant. An uncertain charge needs a different response. If a timeout or crash prevents the runtime

from receiving a reply, the provider may still have charged for the call. Releasing the reservation could leave that charge out of the record. The runtime must keep the reservation until it checks the charge against the provider’s usage records. Only then can it settle the call or release the reservation.

6.3 Exhaustion and truncation

A budget and a per-call limit stop work at different points. The budget prevents a new call from starting. The per-call limit stops generation after a call has started. A run can reach either limit, and the two cases have different consequences.

- **Exhaustion.** Admission refuses the next call, so it never starts. The run ends with a declared status, and a terminal event in the trace records the reason (Table 5). Calls already running continue until they finish, when accounting settles their charges.
- **Truncation.** A call reaches its output limit, so the provider stops generation. The provider returns a successful response containing the content generated so far and reports why generation stopped.

Truncation matters especially when the model requests a tool. The provider’s documentation describes the problem: “If Claude’s response is cut off because it hit the `max_tokens` limit, and the truncated response contains an incomplete tool use block, you’ll need to retry the request with a higher `max_tokens` value to get the full tool use” [1]. A truncated request can still name a known tool and supply arguments that match its signature. For example, an edit request can have valid argument names but incomplete replacement text. The dispatch checks in Section 4.3 test the tool name, number of arguments and policy. They do not check whether the requested content is complete. **The stop reason can flag a potentially incomplete tool request even when it passes every dispatch check.** The runtime must therefore read the stop reason before dispatch. It must refuse an incomplete request before the tool has any effect, then return an observation that the model can use to respond to the failure.

Recovering from truncation costs both the original call and the retry. The truncated call ran and incurred a charge, so accounting settles it as usual. Retrying with a larger output limit starts a new call. That call needs its own admission check and a reservation that covers the larger limit.

Admission also applies to model calls made to retry work, revise a plan or provide a final assessment. If the controller uses the entire budget, no budget remains for assessment, and the run ends exhausted. In the graph implementation of our planning agent, the validator uses the same admission check as the other two model nodes. If a patch passes its checks after the fortieth call, the run ends `budget_exhausted` without a verdict. A design that requires a final assessment sets aside part of the budget for it in advance. Admission then gives the other roles access to B minus that amount. Our code does not implement this reservation.

Accounting must include every role and every attempt. If it counted only controller calls, the planner and validator could spend without reaching the limit. If each attempt received a new budget, the budget would reset after every failed attempt. The settled total must therefore last for the whole run (Section 2.4), just as the call counter does across the proposed attempt edge in Section 3.3. In the graph implementation of our planning agent, all three model nodes update a shared call counter and usage total. Each model-call event in the trace identifies the role that made the call. We can therefore use the trace to count calls by role.

6.4 Controlled measurement

To compare the cost of two implementations, we need a common budget and a controlled design. The scripted comparison described in Section 5.3 tests whether the implementations follow

the same transition rules for the scripted replies. It does not measure their behaviour with a live model. A live model's replies depend on the messages each implementation constructs, so a live comparison measures the model together with the surrounding agent software, called the *harness*: “When we evaluate ‘an agent,’ we’re evaluating the harness and the model working together” [7]. The unit of measurement is a trial.

Definition 6.3 (Trial). “Each attempt at a task is a trial. Because model outputs vary between runs, we run multiple trials to produce more consistent results.”

A comparison changes the implementation while keeping other conditions fixed. It repeats each configuration over several trials. Seven conditions must stay the same.

- **Tasks.** Give every implementation the same alerts in the same test repository.
- **Initial environment.** Start each trial from a fresh copy of the same workspace. This prevents access to files, caches or history from earlier trials only if tools cannot read outside that workspace. Our tools allow such access through the glob argument of `search`, as shown in Section 4.6.
- **Model configuration.** Use the same model, version, sampling parameters and per-call output limit.
- **Prompts.** Give each role the same system prompts and task messages.
- **Tools.** Provide the same tools, with the same signatures, descriptions and limits.
- **Acceptance rules.** Use the same checks and validator so that *accepted* means the same thing for both implementations.
- **Budget policy.** Use the same unit, budget B , and admission rule. With a call-count budget, an implementation that makes more calls uses up the budget sooner. Different budget policies could change the results and make it unclear how much the implementation contributed.

For each implementation, report how often each terminal outcome occurs across trials. Also report i) usage by token kind, ii) calls by role, iii) latency, and iv) the exhaustion rate. An acceptance rate alone leaves unclear whether the other runs were rejected or exhausted their budgets. Usage alone leaves unclear whether a low-cost run completed the task or stopped early.

A fixed seed does not ensure that two runs follow the same sequence of steps. It fixes how the sampler draws from a distribution for a given input, while the input depends on the implementation. If two implementations format messages differently, the model receives different inputs from the first call. A difference in one tool result also changes all later messages. Whether identical requests produce identical outputs depends partly on how the provider serves them, which the seed does not control. Our planning agent uses seed 0 and temperature 0. These settings fix the sampler's configuration but do not determine the sequence of steps in a run. Repeated trials let us measure differences between implementations.

The results support conclusions about the implementations, model, prompts, tasks and budget tested. They do not establish which implementation would perform better with another model or budget. Another model may respond differently to the same messages, and another budget changes where runs stop. A small number of trials may also be insufficient to distinguish a real difference from ordinary variation between runs.

6.5 Reading

Revisit MIT 6.042 [17], §2 (p. 5), for the method used to prove the reservation invariant. Read Grace et al. [7], the sections The structure of an evaluation, How to think about non-determinism in evaluations for agents, and Design the eval harness and graders,

for trials, the harness and isolated environments.

While writing HW1, refer to Anthropic [2] for what the counting endpoint counts and why its result is an estimate. Also refer to Anthropic [1], the sections on `max_tokens`, incomplete tool use blocks and handling truncated responses.

Looking ahead. We have described each agent decision by naming the component that makes it and the artifact that records it. A routing function selects the next location, and a routing event records the choice. The dispatcher permits or refuses a tool request, and an observation records the result. Admission permits a call, and accounting records its cost. The runtime sets the terminal status, and the trace records it. HW1 asks you to describe your agent in the same way. For each decision, identify the responsible component and the artifact that provides evidence of the decision. A reviewer needs both to check what occurred.

References

- [1] Anthropic. Stop reasons and fallback. Claude Platform documentation, <https://platform.claude.com/docs/en/build-with-claude/handling-stop-reasons>, 2026. Accessed 2026-09-24. HW1 reference material, not assigned reading.
- [2] Anthropic. Token counting. Claude Platform documentation, <https://platform.claude.com/docs/en/build-with-claude/token-counting>, 2026. Accessed 2026-09-24. HW1 reference material, not assigned reading.
- [3] Anthropic. Tool use with claude. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>, 2026. Accessed 2026-09-18. HW1 reference material, not assigned reading.
- [4] Cornell CS 2110. 25. Event-Driven Programming. Lecture notes, CS 2110 Object-Oriented Programming and Data Structures, Cornell University, <https://courses.cs.cornell.edu/cs2110/2025fa/lectures/lec25/>, 2025. Fall 2025. Accessed 2026-09-24.
- [5] Cornell CS 2112. Loop invariants. Lecture notes, CS 2112, Cornell University, <https://www.cs.cornell.edu/courses/cs2112/2022fa/lectures/loopinv/>, 2022. Fall 2022. Accessed 2026-09-24.
- [6] CrewAI. Flows. <https://docs.crewai.com/v1.15.22/en/concepts/flows>, 2026. Documentation version 1.15.22. Accessed 2026-09-24. HW1 reference material, not assigned reading.
- [7] Mikaela Grace, Jeremy Hadfield, Rodrigo Olivares, and Jiri De Jonghe. Demystifying evals for AI agents. Anthropic Engineering, <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>, 2026. January 9, 2026. Accessed 2026-09-24.
- [8] Katrina LaCurts. Lecture 20 outline. Lecture outline, 6.033 Computer System Engineering, Massachusetts Institute of Technology, MIT OpenCourseWare, <https://ocw.mit.edu/courses/6-033-computer-system-engineering-spring-2018/pages/week-11/lecture-20-outline/>, 2018. Spring 2018. Accessed 2026-09-24.
- [9] LangChain. LangGraph graph API. <https://docs.langchain.com/oss/python/langgraph/graph-api>, 2026. HW1 setup material, not assigned reading.
- [10] LangChain. Persistence. <https://docs.langchain.com/oss/python/langgraph/persistence>, 2026. Accessed 2026-09-18. HW1 reference material, not assigned reading.
- [11] LangChain. LangGraph runtime. <https://docs.langchain.com/oss/python/langgraph/pregel>, 2026. Accessed 2026-09-24. HW1 reference material, not assigned reading.
- [12] Jukka Lehtosalo. PEP 589: TypedDict, type hints for dictionaries with a fixed set of keys. <https://peps.python.org/pep-0589/>, 2019. Accessed 2026-09-18. Final; incorporated into the typing module.
- [13] LlamaIndex. Agent workflows: Introduction. <https://developers.llamaindex.ai/python/llamaagents/workflows/>, 2026. Accessed 2026-09-24. HW1 reference material, not assigned reading.
- [14] LlamaIndex. Agent workflows: Managing state. https://developers.llamaindex.ai/python/llamaagents/workflows/managing_state/, 2026. Accessed 2026-09-24. HW1 reference material, not assigned reading.
- [15] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data (SIGMOD)*, pages 135–146, 2010. Copy used: <https://15799.courses.cs.cmu.edu/fall2013/static/>

- [papers/p135-malewicz.pdf](#). Accessed 2026-09-24.
- [16] MIT 6.031. Reading 6: Specifications. Course reading, 6.031 Software Construction, Massachusetts Institute of Technology, <https://web.mit.edu/6.031/www/sp21/classes/06-specifications/>, 2021. Spring 2021. Accessed 2026-09-24.
 - [17] MIT 6.042. Lecture 16: State machines and invariants. Lecture notes, 6.042 Mathematics for Computer Science, Massachusetts Institute of Technology, <https://courses.csail.mit.edu/6.042/past-devel/archive/spring03/handouts/lectures/lec16.pdf>, 2003. April 10, 2003. Accessed 2026-09-24.
 - [18] Sanjit A. Seshia. Discrete dynamics: State machines. Lecture slides, EECS 149/249A Introduction to Embedded Systems, UC Berkeley, <https://ptolemy.berkeley.edu/projects/chess/eecs149/lectures/DiscreteDynamics.pdf>, 2015. Slides © 2008–2015 E. A. Lee, A. L. Sangiovanni-Vincentelli, S. A. Seshia. Accessed 2026-09-24.
 - [19] Sanjit A. Seshia. Extended finite state machines, timed automata, hybrid automata. Lecture slides, EECS 149/249A Introduction to Embedded Systems, UC Berkeley, <https://ptolemy.berkeley.edu/projects/chess/eecs149/lectures/ExtendedAndTimedAutomata.pdf>, 2015. Slides © 2008–2015 E. A. Lee, A. L. Sangiovanni-Vincentelli, S. A. Seshia. Accessed 2026-09-24.
 - [20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. ICLR version, arXiv:2210.03629v3.

A Conceptual exercises

For every question, select all correct options. Each question has one or more correct options. Explain why your selected options are correct and why the others are incorrect. The answers and explanations follow in [Appendix B](#).

A.1 How an agent moves through a task

Related material: [Section 1](#).

1. **Same step, different state.** An agent returns to its decision step after reading a file. It is about to run the same decision code as before. Which statements explain how its next action can change?
 - (A) The saved file result can change what the decision step chooses.
 - (B) Returning to the same step restores the saved information to its earlier value.
 - (C) The agent can be at the same step with a different stored conversation and call count.
 - (D) A different next action requires adding a new step to the workflow.
2. **Replies and rules.** Two runs reach the decision step with identical saved information. The model replies differently in the two runs. The program has a fixed rule for choosing the next step from the reply. Which statement best describes the result?
 - (A) The fixed rule forces the model to give the same reply in both runs.
 - (B) The runs may take different routes, although the rule selects one route for each saved state and reply.
 - (C) Different routes show that the rule must choose randomly.
 - (D) The rule assigns a probability to each reply.
3. **Repeating a run.** A team wants to reproduce an agent run exactly. They have the same workflow code and starting information. What else would make the same sequence of decisions possible?
 - (A) The same model replies, while tool results may vary without consequence.
 - (B) The same tool results, while model replies may vary without consequence.
 - (C) A list of all routes the workflow allows, without the results that selected them.
 - (D) The same relevant model replies and tool results in the same order.
4. **Few steps, unbounded state.** A workflow has five kinds of step. It keeps every tool result in a list with no fixed length limit. Which statements follow?
 - (A) The workflow can still have just five kinds of step.
 - (B) The number of distinct configurations can be unbounded.
 - (C) Every new tool result requires a new kind of step.
 - (D) The small number of steps makes it possible to list every reachable configuration in a finite table.
5. **Information needed for the next decision.** Two visits to the controller have the same latest tool result. In one run, two model calls have been completed; in the other, 40 have been completed, reaching the call limit. Which statements explain what the agent must remember?
 - (A) The latest tool result alone is enough to decide whether another model call is allowed.
 - (B) The completed-call total must remain available even when the latest tool result is the same.
 - (C) The program must retain every earlier model reply solely to enforce the numeric call limit.
 - (D) The state should retain past facts that can affect the next decision, without necessarily retaining every detail.
6. **What a tool request changes.** A model reply asks the agent to run a tool. The program checks for that request, sends it to the tool, saves the reply, and counts the completed model call. Which statements are accurate?
 - (A) The model reply is information that arrived during this step.
 - (B) The check for a tool request changes the saved call count.
 - (C) The step sends a request for the tool to perform an external operation.
 - (D) Saving the reply and adding one to the call count change what later steps can read.
7. **Possible paths.** A diagram shows a recorded route in blue and other routes in grey. The grey routes depend on model replies or check results that did not occur in the recorded run. Which interpretation is justified?
 - (A) Each grey route was taken once, but its events were omitted from the log.
 - (B) A grey route is more likely than a blue route because it has fewer steps.
 - (C) The blue route is one actual path among several paths the rules permit.
 - (D) The grey routes become impossible once the blue route is recorded.
8. **What a log can prove.** A log says a file-reading tool succeeded and reports the length of its result. It omits the result text. Some transitions between steps are also absent from the log. Which conclusions are justified?
 - (A) The exact file text can be reconstructed from the reported length.
 - (B) The log can establish that the recorded tool call succeeded.
 - (C) Every saved value after the tool call can be read directly from the log.
 - (D) A missing transition may be inferred from nearby events and the workflow rules, provided it is clearly identified as an inference.
9. **Plan versus workflow.** A planner changes its list of subgoals after seeing repeated tool results. The agent's workflow still has the same named steps. Which statements follow?
 - (A) The new plan can affect later actions when the controller reads it.
 - (B) Each new subgoal becomes a new named step in the workflow.
 - (C) The controller may respond differently even though the workflow structure did not change.
 - (D) The program chooses the next step by directly testing the name of the current subgoal.
10. **A second stall.** The agent has already revised its plan once. Since that revision, three new tool results are identical and show no successful edit. Which statements match the workflow?
 - (A) The earlier results that triggered revision must be deleted before the agent can detect the new pattern.
 - (B) The new repeated results can cause a stop for lack of progress.
 - (C) The plan is revised again each time three results match.
 - (D) The agent can distinguish the new results from the old ones using the saved position where revision occurred.

A.2 What an agent remembers between steps

Related material: [Section 2](#).

11. **A small update.** The controller detects that the call limit has been reached and returns an update that changes only the run's status. What happens to the conversation and observations already saved?
 - (A) They remain in the stored state unless another update changes them.
 - (B) They are removed because the controller did not return them.
 - (C) The controller must copy them into its update to keep them.
 - (D) The status can change without changing the conversation or observation history.
12. **A limit outside the prompt.** The program refuses to make a model call after 40 completed calls. The model is never told the number of calls used. Which statement best explains how the limit can still work?
 - (A) The model will infer the count from the length of the conversation.
 - (B) The model's reply can cancel a call before the call is made.
 - (C) The tool history automatically prevents the 41st model call.
 - (D) The program keeps a count and checks it before every model call.
13. **State after a process stops.** A run stops unexpectedly. Its event log and edited files remain on disk, but the agent saved no checkpoint of its state or pending step. Which statements follow?
 - (A) A new process can resume at the exact pending step from the event log alone.
 - (B) The saved files may show effects of work that the agent already performed.
 - (C) The surviving log is useful for studying the run but does not provide everything needed for resumption.
 - (D) The in-memory conversation and counters necessarily survive because the files do.
14. **Keep, replace, or add.** The agent needs all tool results to detect repetition, only the latest plan for current work, and a total of completed model calls. Which update choices fit those needs?
 - (A) Keep adding new tool results to the saved result list.
 - (B) Replace the old plan when a revised plan becomes current.
 - (C) Add one to the call total for each completed model call.
 - (D) Replace the call total with one after each model call.
15. **Clearing an appended list.** The agent keeps tool results by appending each update to a saved list. A step returns an empty list, intending to clear the history. Which statements are accurate?
 - (A) The saved history stays as it was after appending the empty list.
 - (B) The empty update clears the history because it has no elements.
 - (C) A direct replacement with an empty list would produce the intended reset.
 - (D) The history disappears from the list of declared fields.
16. **A signal used by one step.** A tool step records why progress has stalled so the planner can revise the plan. The planner reads that reason. What should happen next?
 - (A) Leave the reason active until the revised plan succeeds.
 - (B) Clear the reason after the planner uses it.
 - (C) Clear the reason and erase the full tool-result history at the same time.
 - (D) Keep the reason in a growing list and add an empty list after the planner reads it.
17. **New evidence after revision.** Three repeated tool results cause a plan revision. The agent keeps the full result history because other steps need it. How can it judge whether work stalls again after revision?
 - (A) Save the position in the result list where the new plan began.
 - (B) Check only results added after that saved position.
 - (C) Treat the old three results as the first three results after revision.
 - (D) Discard the full history whenever the plan changes.
18. **Retries and a whole-run budget.** Imagine the agent is extended to retry a failed patch. It clears the failed candidate and check results before the retry. Which choices keep a 40-call limit across all attempts?
 - (A) Keep the call total when the candidate and checks are cleared.
 - (B) Reset the call total at each retry because the candidate is new.
 - (C) Check the retained total before each model call, including calls in the retry.
 - (D) Tell the model about the limit in its prompt instead of keeping a total.
19. **Counting every call.** The program stores a completed-call count and refuses a call when the count reaches 40. Which practices are needed to establish that completed calls stay within the limit?
 - (A) Increase the stored count once for each completed call.
 - (B) Check the stored count only in the controller, even though the planner and validator also call the model.
 - (C) Leave the count unchanged on steps that make no model call.
 - (D) Let a plan revision reset the count if it remains below 40 afterward.
20. **Declarations and checks.** An agent uses a Python type annotation to list the names and expected types of its saved fields. An editor checks the annotation, but the program does not validate the values at runtime. Steps may return updates for only some fields. Which statements follow?
 - (A) The annotation can help the editor detect a step that refers to an undeclared field.
 - (B) The annotation alone rejects every wrongly typed value returned while the agent runs.
 - (C) The agent should still start with values for fields its first steps will read.
 - (D) Because updates can be partial, every field in the initial state may safely be omitted.

A.3 Choosing the next action and knowing when to stop

Related material: [Section 3](#).

21. **What the next step knows.** A step reads a search result but saves only the fact that the search finished. The routing check must determine whether the result mentions the flagged line. What information can the check use?
 - (A) The check can use the original result because the earlier step saw it.
 - (B) The check can tell that the search finished, but not whether it mentioned the flagged line.
 - (C) The check could determine whether the flagged line was mentioned if the earlier step had saved that detail.
 - (D) The check can treat the fact that the search finished as evidence that the flagged line appeared in its result.

22. **Two routing conditions hold.** A workflow tests two conditions for continuing. Both conditions hold for one saved state. The program tests them in a fixed order and takes the first match. What follows?
- (A) Both next steps run, since both tests matched.
 - (B) The first matching test determines the next step.
 - (C) The fallback step runs because the two tests conflict.
 - (D) The model's wording determines which test takes priority.
23. **A request and a suggestion.** A model reply says, 'Submit the patch now,' and also requests a tool. The run is still active. The programmer's rule sends replies with tool requests to the tools step. Which statements hold?
- (A) The tool request is a value that affects the routing check.
 - (B) The sentence about submitting overrides the program's rule.
 - (C) The model has changed the rule for future replies.
 - (D) The programmer's rule determines what the request means for the next step.
24. **Different runs, same workflow.** The same compiled workflow handles two alerts. One run revisits the controller many times; the other submits quickly. Which explanations fit?
- (A) Every revisit requires the workflow to be compiled again.
 - (B) Different saved results can make the same routing rule choose different routes.
 - (C) Each visit to the controller is a separate occurrence in the run, even though the named step is the same.
 - (D) Compiling the workflow fixes the exact sequence of steps for every run.
25. **Repairing a patch after a failed check.** A proposed patch fails a mechanical check. The workflow sends the reasons back to the controller so it can try again. Which choices support that repair?
- (A) Keep the failed proposal and the check's reasons available to the controller.
 - (B) Erase the conversation so the controller cannot see the failed proposal.
 - (C) Allow the controller to make a new proposal that replaces the stored patch.
 - (D) Keep the overall call limit in force while repairs continue.
26. **Changing a stalled plan.** Three repeated observations show that the current plan is stalled. The workflow allows one plan revision. Which actions make the revision meaningful?
- (A) Ask the planner to revise the plan without giving the reason for the stall.
 - (B) Give the planner the stall reason, then assess the revised plan using observations collected after the revision.
 - (C) Discard every earlier observation before the planner can revise.
 - (D) Stop the run if observations collected after the revision show another stall.
27. **Counting new attempts.** A proposed extension would start a fresh attempt after a final reviewer rejects a patch. The extension limits both attempts and total model calls. Which information must remain across attempts to enforce those limits?
- (A) The rejected patch must remain the active patch for the next attempt.
 - (B) The old plan must remain active because changing it changes the workflow.
 - (C) The attempt count and completed-call count must survive the reset.
 - (D) A written reflection can replace an attempt count.
28. **Call limits and termination.** A run has a limit on model calls. A tool retry loop can repeat without calling the model, and a model call can wait indefinitely for a reply. Which concerns remain?
- (A) The tool retry needs its own bound or another reason it must stop.
 - (B) The call limit alone makes every individual operation finish.
 - (C) A termination argument also needs operations to return or time out.
 - (D) A successful-patch check ensures the run eventually finds a successful patch.
29. **A narrow stall detector.** Every search returns a different irrelevant result, so no three results are identical. The run still makes no useful progress. What follows?
- (A) A check for identical results can miss this lack of progress.
 - (B) Changing results show that the search is getting closer to a solution.
 - (C) Because the stall check does not fire, the run must eventually succeed.
 - (D) A check before each model call can still refuse new calls when the budget is exhausted.
30. **Interpreting recorded outcomes.** One trace ends with an accepted status. Another ends after a tool result without recording a terminal outcome. A third ends with an error status followed by an exception record. Which conclusions are supported?
- (A) The accepted status proves the patch truly satisfies the task.
 - (B) The accepted status shows the run ended and its acceptance procedure passed.
 - (C) The trace without a terminal outcome does not establish whether the run terminated.
 - (D) The error status identifies the terminal outcome, and the following record identifies the exception.

A.4 Tool requests, execution, and access

Related material: [Section 4](#).

31. **When state updates become visible.** The runtime holds proposed updates until every worker in a batch has finished. Two workers read the saved state at the start of the same batch, and one finishes first. Which statements about saved state follow?
- (A) The other worker reads the state available when the batch began.
 - (B) The other worker sees the first worker's update as soon as that worker finishes.
 - (C) Workers in a later batch can read the combined update after it is applied.
 - (D) If both workers update one field, the runtime can use the update from the worker that finished last without a rule for combining updates.
32. **A file changed before a crash.** A tool edits a file, then its worker crashes before the worker's update to the saved run state is applied. What is possible?
- (A) The file edit is automatically undone with the missing state update.
 - (B) The file has changed even though the saved state does not describe the change.
 - (C) The next worker must see the crashed worker's partial state update.
 - (D) The file change becomes visible only when the worker's saved-state update is applied.

33. **Instructions and enforced limits.** A tool description tells the model to stay inside a workspace, but a search request can still read a sibling folder. Which conclusions follow?
- (A) The instruction in the description cannot itself prevent the tool from reading outside files.
 - (B) The description makes outside reads impossible when the model follows it most of the time.
 - (C) The model can decide whether a particular outside path is permitted by explaining its intent.
 - (D) The actual files reached by search must be checked before they are read.
34. **Who can change a setting.** A new tool takes a run setting as an argument. The dispatcher does not supply that argument, so a model request can supply it. The setting should be controlled by the application. Which change achieves that?
- (A) Ask the model in the tool description to leave the setting unchanged.
 - (B) Give the setting a default value while leaving it available as an argument.
 - (C) Keep the setting outside arguments the model can supply, or have trusted runtime code supply it.
 - (D) Accept the model's value whenever it gives a convincing reason.
35. **Argument validity and access permission.** One request uses the wrong argument name. Another uses the correct name but points outside the allowed workspace. A third supplies a string that can be unambiguously converted to the expected integer type. What follows?
- (A) Checking argument names also proves a path is permitted.
 - (B) The wrong name can be refused before the tool runs, while the path needs an access check before opening the file.
 - (C) If the path argument binds to the tool, the file must be opened.
 - (D) The dispatcher can correct an unambiguous type mismatch without treating it as permission to cross the workspace boundary.
36. **A useful refusal.** An edit request matches two places in a file, and the tool refuses to choose one. How should a continuing run handle this?
- (A) Return a failed result with the reason to the model.
 - (B) Edit the first match so the run makes progress.
 - (C) Make no edit for the refused request.
 - (D) Allow a later request with enough surrounding context to identify one occurrence.
37. **Evidence that a tool ran.** A trace shows a model asked to read a file, the request reached the dispatcher, and a successful result records the number of characters returned. What does this establish?
- (A) The model's request by itself proves the file was read.
 - (B) The successful result supports the conclusion that the reading function ran and returned.
 - (C) The trace does not reveal the exact returned text from its character count.
 - (D) The request record alone shows that all access checks passed.
38. **A new way to read files.** A team has tested that its file-reading tool blocks paths outside the workspace. Later it adds another tool that can read files without using that check. What must the team consider?
- (A) Every route that can read a protected file needs a check before the read.
 - (B) The first tool's test results cover the new tool because both read files.
 - (C) A prompt instruction to remain in the workspace prevents the new tool from reading outside it.
 - (D) A shared access check helps only if every file-reading route uses it.
39. **Resolving a file path.** A workspace is at `/tmp/ws`. A request names a path containing `..`, a symbolic link, or an absolute path. A sibling folder named `/tmp/ws-copy` also exists. Which checks are appropriate?
- (A) Accept any path whose text begins with `/tmp/ws`.
 - (B) Resolve the workspace and candidate to their real destinations before comparing them.
 - (C) Treat a path that starts inside the workspace as potentially outside after resolution.
 - (D) Treat an absolute path as inside the workspace once it has been joined to the workspace path.
40. **One unchecked search route.** A search tool accepts a pattern reaching a sibling folder and returns its lines. The editing tool checks its target path before writing. What can be concluded?
- (A) The write boundary has failed because search changed the matching files.
 - (B) Search can bypass the read boundary.
 - (C) Checking each matched file's resolved destination before reading would close this search route.
 - (D) A successful containment test for the other reading tool does not prove search is contained.

A.5 Events and queued work

Related material: [Section 5](#).

41. **A request is waiting.** A planning assistant puts "run these file checks" in a work queue. The worker has not picked it up. What does the queued item tell us?
- (A) The checks have run because the request is visible in the queue.
 - (B) The worker may run the checks when the runtime delivers that item.
 - (C) The queue has already changed the files being checked.
 - (D) If the request is later delivered, establishing successful execution requires more evidence than the queued request alone.
42. **Application logic and worker scheduling.** A team writes a function to handle a completed plan and registers it with an event loop. Who defines the response, and who decides when to call the function?
- (A) The team decides what the function does when a plan arrives.
 - (B) The function must directly call every worker that might follow it.
 - (C) The runtime decides when to invoke the registered function.
 - (D) The runtime invents the function's rules for updating the plan.

43. **A message with no registered handler.** A worker finishes planning and puts “plan ready” in the queue, but no worker is registered to receive that kind of item. The runtime removes it and the queue becomes empty. What follows?
- (A) The task succeeded because the queue is empty.
 - (B) A planned next step may have been lost.
 - (C) No part of the application necessarily chose to stop.
 - (D) The runtime can recover the missing worker from the plan text.
44. **An idle run without an outcome.** A run has no queued work and no worker running. It also has no recorded outcome. Which conclusion is justified?
- (A) The run succeeded because every queue is now empty.
 - (B) The last worker must have decided to stop.
 - (C) The absence of pending or active work alone does not tell the runtime how the run ended.
 - (D) The run necessarily failed its file checks.
45. **Save before notifying.** A tools worker gets a file result and wants to notify the controller that results are ready. Which practices ensure that the controller reads the intended result?
- (A) Notify the controller first; it can infer the file result from the words “results ready.”
 - (B) Save the file result before the notification can be delivered.
 - (C) It is safe to save after notifying because the queue is empty before the notification.
 - (D) Apply the result once so the controller does not read a duplicate observation.
46. **One message, two deliveries.** A worker has already added one tool result to the saved history. The runtime accidentally delivers the same completion message again. What risk does this create?
- (A) The second delivery proves the tool ran a second time.
 - (B) The first saved result is automatically removed.
 - (C) The controller may act twice on one result or append that result twice.
 - (D) The duplicate makes an explicit stopping outcome unnecessary.
47. **No reply arrives.** A model call should time out after thirty seconds. This event loop starts work only when an item arrives. What makes the timeout path reachable?
- (A) Waiting silently for thirty seconds automatically invokes the timeout worker.
 - (B) A timer can put a timeout item in the queue when the interval ends.
 - (C) A note saying “still waiting” in the log is enough to invoke the worker.
 - (D) The timeout needs some source of a new item if the model never replies.
48. **Work after stopping.** A run declares its outcome, but a “run tools” item remains in the queue. How should the runtime treat it?
- (A) It must start the tools worker before recording the outcome.
 - (B) The pending item alone changes the declared outcome to success.
 - (C) It may ignore the outcome if the queue still contains work.
 - (D) It must prevent that pending item from starting a worker.
49. **One worker or several.** A simple version allows only one pending item or one running worker at a time. A new version allows two workers to run together. What changes in how we describe its progress?
- (A) With one worker at a time, the pending item or running worker identifies the current step.
 - (B) With two workers running, one step name always describes everything that is happening.
 - (C) Allowing two workers means the saved information no longer matters.
 - (D) With two workers running, the workflow may need a description that includes both active steps.
50. **Comparing graph and queue execution.** A team runs the same workflow once by following graph arrows and once by delivering queued items. Both versions run one worker at a time, and the team supplies the same prepared model replies and tool results in the same order. What can comparing saved information after each completed step show?
- (A) Matching observations, file effects, checks, and outcomes support agreement for those prepared inputs.
 - (B) The two versions must take the same amount of clock time.
 - (C) Agreement does not show what a live model would say if the two versions build different messages.
 - (D) The comparison does not cover two workers acting at once.

A.6 Budgets and spending

Related material: [Section 6](#).

51. **Admission and accounting.** A model provider reports the token charge only after a call finishes. The run has a spending limit. What must the runtime do?
- (A) Decide before the call whether it is allowed to start.
 - (B) Wait for the actual charge before deciding whether that same call may start.
 - (C) Add the actual charge to its spending record after the call.
 - (D) Assume every model call has the same token cost.
52. **Spent and reserved amounts.** A run has a limit of 10 units. It has spent 4 and reserved 4 for a call still in progress. A proposed new call could cost up to 3. What follows?
- (A) It may start because only 4 units have been billed so far.
 - (B) It should be refused for now because the amount spent plus the maximum costs of both calls would exceed 10.
 - (C) The unfinished call can be ignored until its final charge is reported.
 - (D) If the unfinished call later costs only 2, the runtime can reconsider the new call.
53. **Estimates and upper bounds.** A provider’s input-token counter gives an estimate, while the request sets an enforced maximum for output tokens. The team wants a strict guarantee that charges never exceed its token budget. Which conclusions follow?
- (A) The input estimate alone guarantees the call cannot cost more than the reserved amount.
 - (B) The reserved amount must cover a justified upper bound on the whole call’s cost.
 - (C) An enforced output limit gives a bound for that part of the call.
 - (D) Past estimates that were accurate once prove all later estimates will be upper bounds.

54. **Two calls at once.** The remaining budget can cover either of two proposed calls, but not both. Two workers check the available amount at the same moment. What prevents both from starting?
- (A) Let each worker check first, then reserve funds later.
 - (B) Start both and correct the record after their charges are reported.
 - (C) Make checking the remaining amount and reserving funds one indivisible action.
 - (D) Give both calls the same output limit.
55. **A call times out.** A model call times out before the runtime receives a reply. The provider might still bill for work already done. What should happen to the funds reserved for that call?
- (A) Release it at once because a missing reply means the provider did no work.
 - (B) Record a zero charge and give the full amount to the next call.
 - (C) Keep the funds reserved until the provider's usage record shows the actual charge.
 - (D) Treat the timeout as a failed attempt and reset all spending for the run.
56. **What does the limit count?** The controller resends a growing conversation on every model call. A team wants the limit to reflect that extra input work. Which statements describe how to account for that input?
- (A) Counting only generated output captures the growing input as well.
 - (B) Counting all processed tokens includes both the sent conversation and generated reply.
 - (C) A money limit may need separate counts for token kinds with different prices.
 - (D) Counting calls fixes the token cost of each call.
57. **An unfinished tool action.** A model reply hits its output limit. It names a valid editing tool and valid arguments, but the replacement text ends partway through. What should the runtime do?
- (A) Run the edit because the tool name and argument names are valid.
 - (B) Inspect why generation stopped before allowing the edit to affect files.
 - (C) Discard the model call's charge because its proposed edit was unusable.
 - (D) Count the original call and check the budget again before a retry with a larger output limit.
58. **Budget for the final assessment.** A run needs one final model call to assess a patch after file checks pass. The controller can otherwise use all available calls first. Which policy protects the final assessment?
- (A) Reserve enough of the run's allowance for the assessment before admitting other calls.
 - (B) Let the controller use the whole allowance and treat assessment as free.
 - (C) Reset the spending total when file checks pass.
 - (D) Skip the admission check for the final call because it comes after the main work.
59. **Spending across attempts.** A future version starts a fresh attempt after a failed patch. It clears the old candidate and checks. Which choices keep the spending limit meaningful across attempts?
- (A) Keep the amount already spent across the whole run.
 - (B) Reset spending along with the candidate so each attempt has a full new allowance.
 - (C) Count planner, controller, and final-assessment calls, including retries.
 - (D) Count only calls that lead to an accepted patch.
60. **Comparing live runs.** A team compares a graph version and a queue-driven version with a live model. Which choices support a useful comparison, and what can it show?
- (A) Run the same tasks with the same model settings, prompts, tools, checks, and budget in both versions.
 - (B) Use one trial per version because the same random seed makes their replies identical.
 - (C) Repeat trials from fresh workspaces, restrict tools to those workspaces, and report outcomes alongside usage and exhaustion rates.
 - (D) Limit the conclusion to the tested model, tasks, prompts, and budget.

B Answers and explanations

The numbers match the questions in [Appendix A](#). Each answer identifies all correct options and explains every choice. Question numbers link back to the exercises.

B.1 How an agent moves through a task

Related material: [Section 1](#).

1. Correct: A, C.

- (A) **Correct.** The decision step reads stored data, so a new observation can change the route it selects.
- (B) **Incorrect.** Revisiting a step does not undo updates made by earlier steps.
- (C) **Correct.** The step name identifies the operation; the stored values distinguish configurations at that step.
- (D) **Incorrect.** The existing step can choose among its existing routes using changed information.

2. Correct: B.

- (A) **Incorrect.** The program's routing rule does not produce or control the external reply.
- (B) **Correct.** Different replies can satisfy different routing conditions. The rule still selects one route for each state and reply.
- (C) **Incorrect.** The difference can come entirely from different replies, without any random choice in the rule.
- (D) **Incorrect.** A routing rule selects an action after a reply arrives. The rule alone does not specify how likely a reply is.

3. Correct: D.

- (A) **Incorrect.** Tool results can enter saved information and change later decisions.
- (B) **Incorrect.** Model replies can request different tools or request submission.
- (C) **Incorrect.** The set of allowed routes does not show which route a run took.
- (D) **Correct.** With the same starting information and the same inputs at each step, the fixed rules select the same routes.

4. Correct: A, B.

- (A) **Correct.** Adding results changes the stored data while leaving the number of named steps unchanged.
- (B) **Correct.** A result list with no length bound can distinguish an unbounded number of configurations.
- (C) **Incorrect.** The existing tool and decision steps can process another result.
- (D) **Incorrect.** An unbounded result list may make that table infinite.

5. Correct: B, D.

- (A) **Incorrect.** The same result occurs in both runs, but the call limit permits another call in only one of them.
- (B) **Correct.** The program needs that total to distinguish a run below the limit from one at the limit.
- (C) **Incorrect.** An accurate running total is enough for this limit; individual replies may be retained for other purposes.
- (D) **Correct.** The next admission check needs the call count. Details that do not affect this decision can be omitted.

6. Correct: A, C, D.

- (A) **Correct.** The program could not derive the particular reply from its saved information alone.
- (B) **Incorrect.** The check decides whether a route applies; the count changes through an update.
- (C) **Correct.** This step issues a request for tool execution. Later handling may check the request before the tool runs.
- (D) **Correct.** Saving the reply and incrementing the count change the information available to later steps.

7. Correct: C.

- (A) **Incorrect.** A possible route is not evidence that it occurred.
- (B) **Incorrect.** The diagram assigns no probabilities, and path length does not supply one.
- (C) **Correct.** The diagram separates the route taken from other permitted continuations.
- (D) **Incorrect.** The grey routes remain possible for other inputs or runs, even though this run took the blue route.

8. Correct: B, D.

- (A) **Incorrect.** Many different texts have the same length.
- (B) **Correct.** Success is one of the facts the event explicitly records.
- (C) **Incorrect.** The log omits the result text and much of the agent's saved information.
- (D) **Correct.** The rules can identify a transition between recorded events, but the log does not record that transition directly.

9. Correct: A, C.

- (A) **Correct.** The plan is passed to the controller, whose replies can then change.
- (B) **Incorrect.** Subgoals are stored task data; the workflow's step names stay fixed.
- (C) **Correct.** The controller's input now includes a revised plan and possibly new observations.
- (D) **Incorrect.** In this agent, routing depends on replies and checks, not on which subgoal is current.

10. Correct: B, D.

- (A) **Incorrect.** The agent keeps the full history and checks only the results since revision.
- (B) **Correct.** After one revision, the stall route ends the run instead of revising again.
- (C) **Incorrect.** The workflow permits one revision; a later stall has a different outcome.
- (D) **Correct.** The saved position marks where the observations collected after revision begin.

B.2 What an agent remembers between steps

Related material: [Section 2](#).

11. Correct: A, D.

- (A) **Correct.** A step that omits a field leaves its stored value in place.
- (B) **Incorrect.** Steps return only fields they change; omission is not a deletion.
- (C) **Incorrect.** The runtime retains omitted fields without asking the node to repeat them.
- (D) **Correct.** The runtime applies the update to the named field and keeps the other stored values.

12. Correct: D.

- (A) **Incorrect.** Conversation length is not a reliable count, and the model does not enforce this limit.
- (B) **Incorrect.** The reply arrives after the call, so it cannot perform the pre-call check.
- (C) **Incorrect.** A history records events but does not itself reject a call.
- (D) **Correct.** The runtime can enforce the limit using saved information that is absent from the model's context.

13. **Correct: B, C.**
- (A) **Incorrect.** The log contains selected events rather than the full working information and pending work.
 - (B) **Correct.** Workspace artifacts can survive even when the agent's in-memory information is lost.
 - (C) **Correct.** The log records selected actions and results. It does not contain the complete state needed to resume the run.
 - (D) **Incorrect.** Files and in-memory information have different lifetimes.
14. **Correct: A, B, C.**
- (A) **Correct.** The repetition check needs earlier results, so replacing the list would discard needed evidence.
 - (B) **Correct.** The new plan supersedes the old one for later work.
 - (C) **Correct.** Each completed call adds one to the total retained for the run.
 - (D) **Incorrect.** Replacing the total with one would discard the count of earlier calls.
15. **Correct: A, C.**
- (A) **Correct.** An empty list contributes no new elements to an append operation.
 - (B) **Incorrect.** Appending zero elements is different from replacing the stored list.
 - (C) **Correct.** A reset must bypass the append rule and write the new list as the stored value.
 - (D) **Incorrect.** An update changes a value; it does not remove the field declaration.
16. **Correct: B.**
- (A) **Incorrect.** A later route could treat the old reason as a new stall before the revised plan has been evaluated.
 - (B) **Correct.** The planner has used the reason to revise the plan. Clearing the reason prevents it from affecting a later routing decision.
 - (C) **Incorrect.** The stall reason should be cleared after use. The tool-result history remains useful for later checks and validation.
 - (D) **Incorrect.** Appending an empty list would leave the old reason in place, so the signal would remain active.
17. **Correct: A, B.**
- (A) **Correct.** The saved position separates results collected before and after the revision.
 - (B) **Correct.** Earlier repeated results should not count as new evidence of another stall.
 - (C) **Incorrect.** Those results came from work under the previous plan. Counting them again would immediately indicate a stall without evidence from the revised plan.
 - (D) **Incorrect.** The saved position lets the stall check use new results while preserving the full history for other steps.
18. **Correct: A, C.**
- (A) **Correct.** The budget applies to the whole run, so earlier calls must still count.
 - (B) **Incorrect.** This could admit more than 40 calls across several attempts.
 - (C) **Correct.** Every place that can issue a call must honor the same limit.
 - (D) **Incorrect.** A prompt does not supply a reliable program check before a call is issued.
19. **Correct: A, C.**
- (A) **Correct.** The stored count must match the calls actually completed for the check to be sound.
 - (B) **Incorrect.** A call from another step could bypass the limit.
 - (C) **Correct.** Changing the count without a completed call would make it inaccurate.
 - (D) **Incorrect.** Resetting the count would exclude earlier calls and could allow the run to exceed its overall limit.
20. **Correct: A, C.**
- (A) **Correct.** The declared field names give the static checker information to compare with the code.
 - (B) **Incorrect.** The editor's static check does not add runtime validation of returned values.
 - (C) **Correct.** A running step may require a value even if the type declaration marks that field as optional.
 - (D) **Incorrect.** Partial updates preserve fields they omit. They do not supply missing values needed by an earlier step.
- B.3 Choosing the next action and knowing when to stop**
- Related material: [Section 3](#).
21. **Correct: B, C.**
- (A) **Incorrect.** The routing check receives the saved state. It cannot read result contents that the earlier step discarded.
 - (B) **Correct.** Search completion is the only fact the earlier step saved for the routing check.
 - (C) **Correct.** The routing check could use that detail if the earlier step had preserved it in the run's state. In this scenario, the detail was discarded.
 - (D) **Incorrect.** Completion says the search ran; it does not say what the search found.
22. **Correct: B.**
- (A) **Incorrect.** A first-match rule selects one next step, even when several tests are true.
 - (B) **Correct.** The program takes the first matching route because the tests have a fixed order.
 - (C) **Incorrect.** The program uses the fallback only when none of the earlier conditions holds.
 - (D) **Incorrect.** The program's test order determines priority. The model's wording does not change that order.
23. **Correct: A, D.**
- (A) **Correct.** The program examines whether the reply contains a request and uses that saved fact to choose a route.
 - (B) **Incorrect.** The rule checks for a tool request. The sentence about submission does not change that check.
 - (C) **Incorrect.** A model reply supplies data to the rule; it does not rewrite the program's routing logic.
 - (D) **Correct.** The rule sends an active reply containing a tool request to the tools step.
24. **Correct: B, C.**
- (A) **Incorrect.** A cycle can revisit a step within one compiled workflow.
 - (B) **Correct.** The rule is fixed, while the values it reads can differ across runs.
 - (C) **Correct.** Each visit to the controller occurs with the state at that point in the run and has its own position in the sequence.
 - (D) **Incorrect.** Compilation fixes the possible routes. Values available at runtime determine which route is selected at each decision.

25. **Correct: A, C, D.**

- (A) **Correct.** The next proposal can use that evidence to address the specific failure.
- (B) **Incorrect.** The repair path retains the conversation, including the failed proposal and reasons.
- (C) **Correct.** A repair needs a new candidate while preserving the evidence that explains the previous failure.
- (D) **Correct.** This repair path has no separate retry counter, so the run's call limit bounds repeated attempts.

26. **Correct: B, D.**

- (A) **Incorrect.** Without the recorded reason, the planner loses evidence of what failed and may repeat the same approach.
- (B) **Correct.** The stall reason lets the planner respond to the failure. Later stall checks assess observations collected under the revised plan.
- (C) **Incorrect.** The workflow retains all observations and changes the starting position of the results used by the stall check.
- (D) **Correct.** Only one revision is allowed, so a second stall ends the run instead of starting another revision.

27. **Correct: C.**

- (A) **Incorrect.** A fresh attempt must clear the old patch so it is not mistaken for the new candidate.
- (B) **Incorrect.** Plans are run data; replacing one does not change the workflow's structure.
- (C) **Correct.** Resetting either count would exclude earlier attempts or calls and could allow the run to exceed the corresponding limit.
- (D) **Incorrect.** A reflection explains the failure but does not measure how many attempts have already been used.

28. **Correct: A, C.**

- (A) **Correct.** The tool retry loop does not make model calls, so it does not increase the model-call count.
- (B) **Incorrect.** A limit on the number of calls says nothing about the duration of one call or tool operation.
- (C) **Correct.** A decreasing count bounds repeated cycles only if each operation between decreases finishes.
- (D) **Incorrect.** The check recognizes success when it occurs; it cannot force a future patch to pass.

29. **Correct: A, D.**

- (A) **Correct.** The search results are irrelevant but differ from one another, so they do not satisfy the identical-result condition.
- (B) **Incorrect.** Difference alone says nothing about relevance to the task.
- (C) **Incorrect.** A detector for one failure pattern supplies no success or termination guarantee for other patterns.
- (D) **Correct.** The budget applies to calls regardless of whether a stall pattern is detected.

30. **Correct: B, C, D.**

- (A) **Incorrect.** The accepted status shows that the acceptance procedure passed. That procedure can still miss a defect in the patch.
- (B) **Correct.** An accepting terminal event records both termination and acceptance by the procedure.
- (C) **Correct.** The incomplete trace could come from an active run or from a process that stopped before recording a terminal event.
- (D) **Correct.** The terminal status identifies the outcome, while the following record identifies the exception.

B.4 Tool requests, execution, and access

Related material: [Section 4](#).

31. **Correct: A, C.**

- (A) **Correct.** The other worker reads the state from the start of the batch. The runtime applies updates only after all workers in that batch finish.
- (B) **Incorrect.** Updates remain invisible to other workers in the same batch until the batch's update phase.
- (C) **Correct.** The runtime applies the batch's writes before planning later work.
- (D) **Incorrect.** Competing writes need a specified way to combine them or must be rejected.

32. **Correct: B.**

- (A) **Incorrect.** The rule for applying state updates does not undo file changes already made by a tool.
- (B) **Correct.** The external edit can happen during execution before the worker returns a state update.
- (C) **Incorrect.** A worker's writes become visible only when the runtime applies its completed update.
- (D) **Incorrect.** A file edit is an external effect and can occur before any update to saved run state is applied.

33. **Correct: A, D.**

- (A) **Correct.** Text shown to the model may influence its requests, but the execution path needs an access check to enforce the boundary.
- (B) **Incorrect.** A boundary must handle any request that reaches the tool, including a mistaken or hostile one.
- (C) **Incorrect.** The runtime's policy determines access permission. The model's explanation does not grant permission.
- (D) **Correct.** Checking each file before reading it enforces the boundary that the description alone cannot enforce.

34. **Correct: C.**

- (A) **Incorrect.** The model can still supply the argument despite an instruction to leave it unchanged.
- (B) **Incorrect.** A caller can override a default if the dispatcher still accepts a value for that argument.
- (C) **Correct.** The model then cannot replace the run setting through its arguments.
- (D) **Incorrect.** A justification in the request does not transfer authority over trusted configuration.

35. **Correct: B, D.**

- (A) **Incorrect.** A path supplied under the correct argument name can still refer to a forbidden location.
- (B) **Correct.** Name binding and file permission are separate checks at different stages of execution.
- (C) **Incorrect.** Binding establishes that an argument fits the signature; the tool can still refuse it under the path policy.
- (D) **Correct.** Type correction handles a representation mistake; it does not change the access policy.

36. **Correct: A, C, D.**

- (A) **Correct.** The reason tells the model why the request failed and what to clarify.
- (B) **Incorrect.** The request did not identify which occurrence to change, so picking one risks an unintended edit.
- (C) **Correct.** The tool must refuse the request before making an edit, leaving the file unchanged.
- (D) **Correct.** A recoverable refusal lets the model refine its request and continue the run.

37. **Correct: B, C.**

- (A) **Incorrect.** A request can be refused or fail before the tool executes.
- (B) **Correct.** The successful result records that the dispatched read completed.
- (C) **Correct.** The character count records the length of the returned text without revealing its contents.
- (D) **Incorrect.** The request record precedes the checks; their outcome appears in the result.

38. **Correct: A, D.**

- (A) **Correct.** The new route can bypass the boundary because it does not use the tested check.
- (B) **Incorrect.** The new tool follows a different execution path and may skip the tested check.
- (C) **Incorrect.** The instruction may influence requests but does not enforce access at execution time.
- (D) **Correct.** A shared check enforces access only if no file-reading route bypasses it.

39. **Correct: B, C.**

- (A) **Incorrect.** The prefix /tmp/ws also matches /tmp/ws-copy, and links or '..' can change the actual destination.
- (B) **Correct.** Resolving the paths accounts for parent-directory segments and symbolic links before the runtime checks containment.
- (C) **Correct.** A link or parent-directory segment can resolve outside the workspace even when the path text appears to refer to a file inside it.
- (D) **Incorrect.** Joining an absolute path can leave it absolute; its resolved destination still needs the containment check.

40. **Correct: B, C, D.**

- (A) **Incorrect.** Search reads matching files; the editing tool is the writer and checks its target.
- (B) **Correct.** Search can return lines from outside the workspace because it does not check the location of each matched file.
- (C) **Correct.** The check would refuse a match whose real location is outside the workspace before its contents are read.
- (D) **Correct.** The two tools use different execution paths to access files. A passing test for one tool does not establish how the other behaves.

B.5 Events and queued work

Related material: [Section 5](#).

41. **Correct: B, D.**

- (A) **Incorrect.** A queued request records intended work. The worker has not yet executed it.
- (B) **Correct.** The runtime starts the computation by delivering the request to a worker registered for that kind of work.
- (C) **Incorrect.** Putting an item in a queue does not perform the worker's file operations.
- (D) **Correct.** An execution record or a successful result can show that the checks ran. The queued request records only the intended work. A refusal result would show that the checks did not run.

42. **Correct: A, C.**

- (A) **Correct.** Application code contains the steps that read the plan, update saved information, and choose what work comes next.
- (B) **Incorrect.** The function can put a new item in the queue; the runtime delivers it to the registered worker.
- (C) **Correct.** The loop checks pending items and calls the function registered for their kind.
- (D) **Incorrect.** Those rules come from the team's code, even though the runtime controls invocation.

43. **Correct: B, C.**

- (A) **Incorrect.** An empty queue only says that no work is scheduled; it does not record a successful outcome.
- (B) **Correct.** The completed plan was supposed to trigger another computation, but no worker is registered to receive the message.
- (C) **Correct.** Work can cease because delivery failed, without a worker making a stopping decision.
- (D) **Incorrect.** The runtime dispatches each item according to its registered handler. No handler is registered for this kind of item.

44. **Correct: C.**

- (A) **Incorrect.** A queue can be empty because the run finished intentionally or because a step failed to schedule further work.
- (B) **Incorrect.** The last worker could have failed to schedule its continuation.
- (C) **Correct.** A recorded outcome is needed to distinguish among success, an intentional stop, and a failure to schedule the next step.
- (D) **Incorrect.** Queue state says nothing about the result of checks that may or may not have run.

45. **Correct: B, D.**

- (A) **Incorrect.** The notification identifies the next kind of work but does not contain the full file result.
- (B) **Correct.** The controller then reads the updated information when it starts.
- (C) **Incorrect.** The runtime may deliver the notification immediately, before the result is saved.
- (D) **Correct.** Repeating the same update could make one tool result appear twice in the saved history.

46. **Correct: C.**

- (A) **Incorrect.** Repeating a message is a delivery error and does not establish a second tool execution.
- (B) **Incorrect.** A repeated message does not undo an append to history.
- (C) **Correct.** Two deliveries can cause the controller to process the same result twice.
- (D) **Incorrect.** Duplicate delivery does not explain when or why the run should end.

47. **Correct: B, D.**

- (A) **Incorrect.** Waiting without an arriving item does not trigger work in this event loop.
- (B) **Correct.** The loop can deliver the timer event to the timeout worker.
- (C) **Incorrect.** A log entry records an occurrence but is not necessarily an item submitted for dispatch.
- (D) **Correct.** Without a reply or timer event, no input arrives to start the timeout transition.

48. **Correct: D.**

- (A) **Incorrect.** The decision to end the run should prevent further work from starting.
- (B) **Incorrect.** An item requesting work does not determine the status of a run.
- (C) **Incorrect.** A declared stop is the point after which no new worker should start.
- (D) **Correct.** The terminal decision stops further dispatch even if items remain in the queue.

49. **Correct: A, D.**

- (A) **Correct.** The single pending item or running worker identifies one step in the original workflow.
- (B) **Incorrect.** The workers may be at different parts of the workflow at the same time.
- (C) **Incorrect.** Both workers still read and update saved information, and their timing now matters more.
- (D) **Correct.** A single current-step label cannot represent simultaneous computations at different locations.

50. **Correct: A, C, D.**

- (A) **Correct.** The observations, file effects, checks, and outcomes are observable results of the transition rules at each comparison point.
- (B) **Incorrect.** The comparison fixes inputs and step boundaries, not scheduling overhead or timing.
- (C) **Correct.** Different messages can lead a live model to give different replies even when the step rules match.
- (D) **Correct.** The comparison uses one worker at a time. Cases with concurrent workers require separate analysis.

B.6 Budgets and spending

Related material: [Section 6](#).

51. **Correct: A, C.**

- (A) **Correct.** To prevent a charge that would exceed the limit, the runtime must check admission before the call begins.
- (B) **Incorrect.** The actual charge is not yet known, so it cannot determine whether that same call may start.
- (C) **Correct.** Later admission decisions need an accurate total of charges already incurred.
- (D) **Incorrect.** The input grows with the conversation, and outputs can have different lengths.

52. **Correct: B, D.**

- (A) **Incorrect.** The runtime must also count the 4 units reserved for the unfinished call.
- (B) **Correct.** Starting the new call would commit up to 11 units in total: 4 spent, 4 reserved, and 3 more.
- (C) **Incorrect.** Ignoring the unfinished call would let multiple calls use the same remaining allowance.
- (D) **Correct.** Releasing the unused 2 units would leave enough budget to reserve 3 for a later call.

53. **Correct: B, C.**

- (A) **Incorrect.** The provider warns that actual input usage can differ from the estimate.
- (B) **Correct.** If the final charge can exceed the reserved amount, the run can overspend when the call finishes.
- (C) **Correct.** Generation stops at the requested maximum, so output use cannot exceed that limit.
- (D) **Incorrect.** Past accuracy can inform an estimate, but it does not establish an upper bound for every future request.

54. **Correct: C.**

- (A) **Incorrect.** Both may see the same available amount and each may decide that its call fits.
- (B) **Incorrect.** The spending limit could already have been exceeded by then.
- (C) **Correct.** The second worker then sees the first worker's reservation before deciding whether its own call fits the budget.
- (D) **Incorrect.** Equal output limits do not ensure that the combined cost fits within the remaining budget.

55. **Correct: C.**

- (A) **Incorrect.** The provider may have processed part or all of the request before the timeout.
- (B) **Incorrect.** Recording a zero charge is unjustified while the actual charge is unknown.
- (C) **Correct.** The funds remain reserved until the runtime can establish the charge or confirm that there was none.
- (D) **Incorrect.** A failed attempt can still incur a charge, and earlier charges also remain part of the run.

56. **Correct: B, C.**

- (A) **Incorrect.** Output-only counting excludes the tokens sent into the model.
- (B) **Correct.** Counting all processed tokens includes both input and output tokens.
- (C) **Correct.** Fresh input, cache reads, and output can have different prices before being combined into one charge.
- (D) **Incorrect.** One call can resend a much longer conversation than another.

57. **Correct: B, D.**

- (A) **Incorrect.** Valid tool and argument names do not establish that the replacement text is complete enough to apply safely.
- (B) **Correct.** A stop caused by the output limit indicates that a seemingly valid request may be incomplete.
- (C) **Incorrect.** The model call still ran and consumed provider resources.
- (D) **Correct.** A retry is another paid call. The runtime must check whether the budget can cover its larger possible output.

58. **Correct: A.**

- (A) **Correct.** Other roles cannot use the allowance reserved for the final assessment.
- (B) **Incorrect.** The assessment is a model call and incurs usage too.
- (C) **Incorrect.** Earlier calls remain charged even after the patch reaches assessment.
- (D) **Incorrect.** The final call still needs an admission check under the same run limit.

59. **Correct: A, C.**

- (A) **Correct.** Clearing a failed candidate must not erase charges incurred while producing it.
- (B) **Incorrect.** Repeated attempts could then exceed the stated limit for the run.
- (C) **Correct.** Calls from every model role can incur charges. Omitting a role would understate the amount spent on the run.
- (D) **Incorrect.** Failed calls and rejected attempts still consume resources.

60. **Correct: A, C, D.**

- (A) **Correct.** Holding these conditions fixed helps isolate the effect of how work is scheduled.
- (B) **Incorrect.** A seed does not make different model inputs produce the same replies, and one trial cannot show variation.
- (C) **Correct.** Keeping tools inside each fresh workspace blocks access to files left by earlier trials. Outcomes and costs show how and why runs stopped.
- (D) **Correct.** Another model or budget could change both the trajectory and the result.